Werknorm / Company Standard

Software Coding Standard for the C Programming Language

CONFIDENTIAL LEVEL: ☒**CONFIDENTIAL** ☐**SECRET**

EXTERNAL DIST: ☐**ALLOWED** ☒**NOT ALLOWED**

SIGNATURE-TYPE	ROLE	NAME	DATE	SIGNATURE
Prepare	Creator	Elmar Ihler	11.06.2019	Electronic Signature
Verify	Standardization Manager	Klaus Wulz	05.07.2019	Electronic Signature
Approve	Process Owner	Ulrich Zanker	08.07.2019	Electronic Signature
Authorize	QM Representative	Juergen Pempera	10.07.2019	Electronic Signature

The following DO-178B/DO-178C / ED-12B/ED-12C aspects are covered by the chapters 7-8 of this document (detailed mapping of rules see section 12):

DO-178B/C / ED-12B/C Reference	Objective
11.8a	Programming language(s) to be used and/or defined subset(s). For a programming language, reference the data that unambiguously defines the syntax, the control behaviour, the data behaviour and side-effects of the language. This may require limiting the use of some features of a language.
11.8b	Source Code presentation standards, for example, line length restriction, indentation, and blank line usage and Source Code documentation standards, for example, name of author, revision history, inputs and outputs, and affected global data.
11.8c	Naming conventions for components, subprograms, variables, and constants.
11.8d	Conditions and constraints imposed on permitted coding conventions, such as the degree of coupling between software components and the complexity of logical or numerical expressions and rationale for their use.
11.8e	Constraints on the use of the coding tools.

Deskriptor / Descriptor:
Software, RTCA/DO-178B, RTCA/DO-178C, Software Coding Standards

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

Liebherr-Aerospace Lindenberg GmbH

Pfänderstraße 50-52 - 88161 Lindenberg im Allgäu / Germany

Table of Contents

1 Purpose 7
2 Scope and Application 7
3 Terms and Abbreviations 8
3.1 Terms 8
3.2 Acronyms and Abbreviations 9
4 Documents 10
4.1 Mandatory Documents 10
4.2 Supporting Documents 10
4.3 Relationship to Other Plans and Standards 11
5 Responsibilities 12
6 Applying the Coding Conventions 12
6.1 Rule and Guideline Categories 12
6.2 Description Format of a Rule 13
6.3 Description Format of a Guideline 13
6.4 Use and Definition of “required” and “advisory” 13
6.5 Use and Definition of the Words “must”, “should” and “may” 14
6.6 Compliancy Check 14
6.7 Annotating the Source Code 14
7 Rules 16
7.1 Code Layout 16
7.1.1 Comments Style 16
7.1.2 No Nested Comments 17
7.1.3 “One-Statement-Block” Block Borders 18
7.1.4 One Statement per Line 19
7.1.5 Declaration of Variables on Separate Lines 20
7.2 Environment 21
7.2.1 Floating-Point Implementations 21
7.2.2 Programming According to ANSI C Definition 22
7.2.3 Reliance in Undefined and Unspecified Behavior 23
7.2.4 Implementation-Specific Behaviours 24
7.2.5 Characters Sets 25
7.3 Programming Techniques 27
7.3.1 Naming Conventions 27

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.1.1 Filenames ... 27
7.3.1.2 Overlapping Identifier Name of Outer Scope ... 28
7.3.1.3 Significance of Identifiers within the First 63 Chars ... 29
7.3.2 Types, Declaration and Definition ... 31
7.3.2.1 Function Prototyping ... 31
7.3.2.2 Macro Definition is Restricted ... 33
7.3.2.3 Undefine of Macro is Not Allowed ... 35
7.3.2.4 Scope of Macro Definition ... 36
7.3.2.5 Function-Like Macros in Handwritten Code ... 37
7.3.2.6 Function-Like Macros in Generated Code ... 41
7.3.2.7 Use of Numeric Constants ... 43
7.3.2.8 Octal Constants ... 44
7.3.2.9 Basic Types ... 45
7.3.2.10 Usage of Static Variables ... 46
7.3.2.11 Array Declaration ... 47
7.3.2.12 Array Initialisation ... 48
7.3.2.13 Array Index Type ... 49
7.3.2.14 Array Element Control Index ... 50
7.3.2.15 Implicit Type Conversions ... 53
7.3.2.16 Explicit Type Conversions with the "cast" Operator ... 55
7.3.2.17 Casting of Pointers ... 57
7.3.2.18 Variable and Pointer Initialization Usage ... 59
7.3.2.19 Complexity of Pointer Constructions Limited to 2 ... 61
7.3.2.20 Explicit Type for Declaration of Variables and Functions ... 62
7.3.2.21 Enum Initialization ... 63
7.3.2.22 Bitfields Types ... 64
7.3.3 Statements ... 65
7.3.3.1 Dead Code ... 65
7.3.3.2 Conditions with Invariant Result ... 66
7.3.3.3 Trigraphs ... 67
7.3.3.4 Use of "goto" and Labels ... 68
7.3.3.5 Embedding Assembler Code into C Code ... 70
7.3.3.6 Logical Expressions Enclosed in Brackets ... 72
7.3.3.7 Use Restriction of Assignment Statement ... 73
7.3.3.8 Test against Zero ... 74
7.3.3.9 "switch" Statement Scope for "case" and "default" ... 75

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3.10 Use of “break” in “switch / case” Branches ... 77
7.3.3.11 Minimal "switch" Statement ... 78
7.3.3.12 "switch" Expression as Boolean Value ... 79
7.3.3.13 Usage and Structure of "if-else if"-Conditions ... 80
7.3.3.14 Use of "for" Loop Counter ... 81
7.3.3.15 "for" Loop Must Have an End Condition ... 82
7.3.3.16 Variable Used as End Border in "for" Loop ... 83
7.3.3.17 Expressions Used in "for" Loop Statements ... 85
7.3.3.18 Change of Loop Counter of "for" Loop ... 86
7.3.3.19 No Floating Point Type for Counter of "for" Loop ... 87
7.3.3.20 Usage of Break and Continue Statements in Iterative Structures ... 88
7.3.3.21 Only One Side Effect or Control Flow Change per Non-Null Statement ... 90
7.3.3.22 NULL Statements ... 94
7.3.3.23 Pointer Arithmetic ... 96
7.3.3.24 Pointer Comparison ... 98
7.3.3.25 Function Pointers ... 99
7.3.3.26 Structs and Unions Members ... 100
7.3.3.27 Floating Point Comparisons ... 101
7.3.3.28 Floating Point Representation ... 103
7.3.3.29 Preprocessor Statement “#include” ... 105
7.3.3.30 Preprocessor Statement “defined” ... 106
7.3.3.31 Preprocessor Operators # and ## ... 107
7.3.3.32 Preprocessor Directive with Identifiers ... 108
7.3.3.33 Preprocessor Directives Always Valid ... 109
7.3.3.34 Preprocessor Directives “#else”, “#elif”, “#endif” ... 110
7.3.4 Function Usage Restrictions ... 111
7.3.4.1 Declaration of Function Parameters ... 111
7.3.4.2 Needs of "return" Statement in Functions ... 113
7.3.4.3 Number of "return" Statements in a Function ... 115
7.3.4.4 Return Value of “void” Functions ... 117
7.3.4.5 Function Call Parameters ... 118
7.3.4.6 Function Call without Defined Parameters ... 119
7.3.5 Include File ... 120
7.3.5.1 Definition and Declaration of Functions or Objects in Include Files ... 120
7.3.5.2 “Typedefs” in Include Files ... 122
7.3.5.3 Recursive/Repetitive Inclusion of Include Files Content ... 124

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.6 Operators 126
7.3.6.1 Forbidden Operators 126
7.3.6.2 Use of “? :” Operator 127
7.3.6.3 Use of the “unary minus” Operator 128
7.3.6.4 Use of the “comma” Operator 129
7.3.6.5 Use of the “shift” Operator 130
7.3.6.6 Use of the “sizeof” Operator 131
7.3.6.7 Bitwise Operations Must Be cast 132
7.3.6.8 Bitwise Operations Only On Unsigned Integer 133
7.3.6.9 Logical Operators and Side Effects 134
7.3.6.10 Logical Operators Need Primary Expressions 135
7.3.6.11 Assignment Operators 136
7.3.6.12 Operator Precedence 137
8 Guidelines 138
8.1 Layout of Source File Headers 138
8.2 Indent Every New Block with 4 Blanks (Spaces) 143
8.3 Indentation with Tabulator 144
8.4 Horizontal Spacing 145
8.5 Pointer to Constant as Function Parameter 147
8.6 Constant Pointer as Function Parameter 148
8.7 Expression Bracket Depth Exceeding 7 149
8.8 Prefixes and Postfixes of Identifiers 150
8.9 Location of Function Prototypes 152
8.10 Operation Naming Conventions for OOP 154
8.11 Maximum of 150 Characters per Line 155
8.12 Storage Type "register" 156
8.13 Maximum Block Depth Limited to 6 157
8.14 Valid Pointers 159
8.15 No Source Code “Commented Out” 160
8.16 Overflow of Unsigned Integer with Constant 161
8.17 Complex Type Definition with Tag 162
8.18 Preprocessor Directive “#endif” with Comments 163
8.19 Definition of Basic Types 164
8.20 Floating Point Integration 166
8.21 Separate Lines for the Block Borders "{" And "}" 167
8.22 Comparison Operators for End Condition In "for" Loop 168

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

8.23 Naming Convention of Identifiers 169
8.24 Type of a “#define” Value 171
8.25 Active Code Hidden in a “#define” 172
8.26 No Calculation Expression at Function Call 173
9 Amendment Record 174
10 APPENDIX 1: Cross-Reference Matrix to MISRA-C 175
11 APPENDIX 2: Rules / Guidelines Summary 187
12 APPENDIX 3: DO-178B/DO-178C / ED-12B/ED-12C Objectives Coverage 192

List of Tables

Tab. 1 Mandatory Documents 10
Tab. 2 Supporting Documents 10

List of Figures

Fig. 1 Relationship to other Plans and Standards 11
Fig. 2 Rule and Guideline Categories 12

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

1 Purpose

The purpose of this company standard is to specify software coding conventions and language dependent requirements for software coding using the C programming language. This document has been written under full consideration of the objectives defined in the RTCA DO-178B/DO-178C / ED-12B/ED-12C chapter 11.8.

This software standard defines conventions concerning:

- Layout, format and indentation
- Uniform presentation of information throughout the source code
- Use of headers
- Size of code aggregates
- Naming conventions
- Implementation restrictions
- Use of language constructs and features
- Reducing code complexity
- Increase reliability and maintainability
- Support portability requirements

2 Scope and Application

This company standard shall be applied to all software products implemented in the C Programming Language by the Liebherr-Aerospace Lindenberg Software department.

This standard may be expanded by additional agreements with subcontractors and/or project specific standards.

This revision of these standards is binding from the moment of publication for new projects starting after publication. Projects already started prior to publication can decide depending on specific project needs if a change to this revision of these standards is feasible.

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

3 Terms and Abbreviations

3.1 Terms

Class - In OOP a class is an element (similar to a C type structure) containing the definition of data and functions that are closely related to each other.

Dead Code – (from [DO-178C]) Executable Object Code (or data) which exists as a result of a software development error but cannot be executed (code) or used (data) in any operational configuration of the target computer environment. It is not traceable to a system or software requirement. The following exceptions are often mistakenly categorized as dead code but are necessary for implementation of the requirements/design: embedded identifiers, defensive programming structures to improve robustness, and deactivated code such as unused library functions.

Function - Basic unit of C code.

MISRA-C - Coding guidelines for C that have originally been developed by the UK automotive industry and are now in use across automotive, aerospace, medical and other industries.

Module - A small section of a program that carries out a clearly defined task. It regroups features that are closely related together in order to be deployed as one component, like a driver containing several access functions. It is designed to be combined with other modules and is reusable for different programs, but it cannot be deployed piecemeal.

A module could be implemented in different ways depending of the design description method used and the programming language, but it must have in any case a public interface which contains only the software elements needed to be connected to it.

For the C language, it could be constituted in:

- one or more .h files, one of them containing the public interface of the module
- zero or more .c files, the module should have passive software elements only.

Operation - OOP Code that is part of a class. An operation corresponds to a C-function.

Scope - The scope of a name is the part of the program in which the name is valid and can be used.

Side effect - Function calls, nested assignments and the increment- and decrement- operators cause “side effects” - additional to the calculation of an expression, a variable will be modified.

Software Design Description Method - For the development and description of software architecture and software low-level requirements three different methods and associated tools are permitted (see [SDS] for more details):

- **SDDM1**: UML based software design, using the tool Ameos
- **SDDM2**: Functional Decomposition, using the tool SCADE
- **SDDM3**: Functional Decomposition, using the tool Matlab/Simulink

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

3.2 Acronyms and Abbreviations

ANSI	American National Standards Institute
CPU	Central Processing Unit
DAL	Design Assurance Level
ID	Identifier
IEEE	Institute of Electrical and Electronics Engineers
LAT	Liebherr-Aerotechnik Lindenberg (company name till 30.06.1996)
LLI	Liebherr-Aerospace Lindenberg (new company name)
MISRA	Motor Industries Software Reliability Association
OOP	Object Oriented Programming
OPR	Open Problem Report
PSAC	Plan for Software Aspects of Certification
QA	Quality Assurance
RTCA	Radio Technical Commission for Aeronautics
SCADE	Saftey Critical Application Development Environment (Design Modeling-Tool, Functional Decomposition Principle)
SCMP	Software Configuration Management Plan.
SCS	Software Coding Standard document of LLI (this document)
SDDM1 SDDM2 SDDM3	Software Design Description Method ‘x’. For detailed description of each method, see in the [SDS].
SDRR	Software Development Results Report
SDS	Software Design Standard document of LLI (LAT9-3029)
SVP	Software Verification Plan
SW	Software
UK	United Kingdom
UML	Unified Modeling Language

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

4 Documents

4.1 Mandatory Documents

Documents listed below contain mandatory guidelines/provisions which are referred to in the text of this company standard. These documents must be available to subcontractors for outsourced manufacturing.

Important note:
National/international standards have to be procured by subcontractors itself due to licensing reasons.
Before procuring a national/international standard the contents of this company standard must be checked. Depending on the scope of this company standard, it is possible that not all mandatory documents are required.

Document	Title
[DO-178B]	Software Considerations in Airborne Systems and Equipment Certification RTCA/DO-178 B / ED-12B
[DO-178C]	Software Considerations in Airborne Systems and Equipment Certification RTCA/DO-178 C / ED-12C
[SCMP]	Software Configuration Management Plan. Should be LAT9 0202, or can be a project specific one.
[SDS]	Software Design Standard: LAT9-3029
[SVP]	Software Verification Plan. Should be LAT9-0204, or can be a project specific one.

Tab. 1 Mandatory Documents

4.2 Supporting Documents

Supporting documents are only relevant within LLI and are not required for application of this company standard.
They do not include guidelines/provisions, but refer to related information or sources.
Supporting documents are not made available to subcontractors.

Document	Title
[ANSI-C90]	American National Standards Institute, American National Standard for Information Systems - Programming Language C (ANSI C) ANSI/ISO 9899-1990 (C90)
[IEEE-754]	IEEE Standard for Binary Floating-Point Arithmetic for microprocessor systems ANSI / IEEE Std 754-2008
[MISRA-C]	Motor Industry Software Reliability Association: Guidelines for the use of the C language in critical systems MISRA-C: 2004
[ISO 10646-1]	Information technology -- Universal Coded Character Set (UCS) ISO/IEC 10646:2017

Tab. 2 Supporting Documents

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

4.3 Relationship to Other Plans and Standards

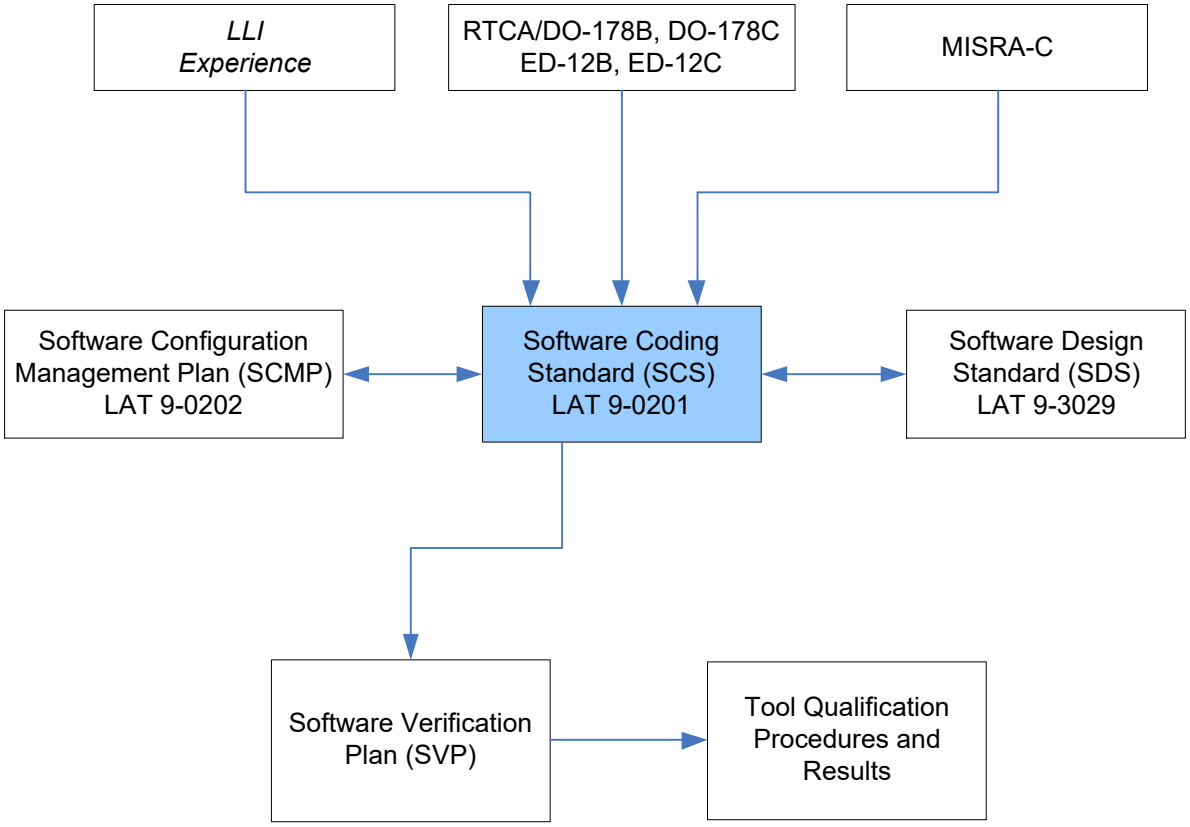


Fig. 1 Relationship to other Plans and Standards

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

5 Responsibilities

Each *Designer Software* is responsible for ensuring that:

a) the source code conforms to this standard
b) when any static test tool is used, the specific configuration used to configure the rules and the output of the check tool is identified, stored, versioned and checked for adequacy with the rules defined in the coding standard
c) The version and the revision history of each source code file are managed according to the Software Configuration Management Plan [SCMP], including the process and the tool expected in the project PSAC. This acts as compliancy means to [DO-178B]/[DO-178C] objective 11.8b.
d) all rule violations are justified (required rules) and listed (advisory rules)

Additionally the *Software Engineer* is responsible for ensuring that the entire SW-Team:

a) strictly follows the rules of this standard
b) obeys the guidelines of this standard as much as possible
c) uses source code annotations only in the manner specified in chapter 6.7
d) manages the source code according to the Software Configuration Management Plan [SCMP] specially concerning the versioning of the Source Code files

For rule and guideline categories refer to chapter 6.1.

6 Applying the Coding Conventions

This chapter contains information about the description and utilization of the coding conventions. Details about checking the correct function of the code is not part of this document.

The coding conventions are organized in two main categories: the Rules and the Guidelines.

6.1 Rule and Guideline Categories

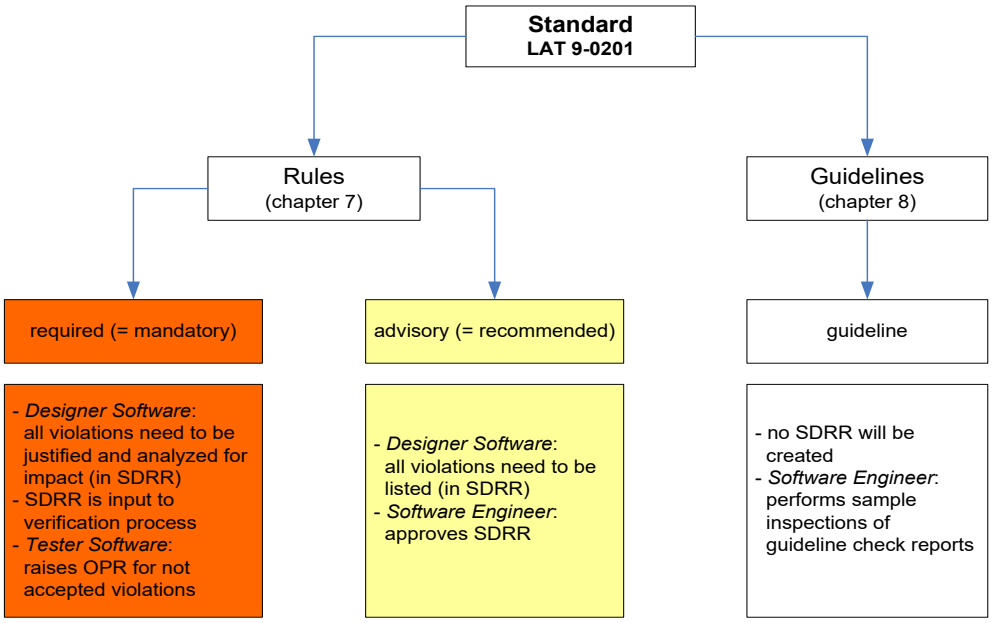


Fig. 2 Rule and Guideline Categories

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

6.2 Description Format of a Rule

Each rule has a title and seven description sections:

1) **ID**: The unique ID of the coding convention. These assigned IDs will not be changed with future releases of this standard.
2) **Rule**: The detailed requirement of the rule and specifically, what is and what is not 'allowed'.
3) **DAL**: Depending on the software being assessed, the software Design Assurance Level that the rule applies to with reference to [DO-178B] / [DO-178C] (reference section 3). For every level, the importance is specified (required / advisory).
4) **Reason**: A brief explanation and the rationale behind why the rule is required and its positive/negative effect(s) and/or impact.
5) **Applicability**: If the rule cannot be applied to all methods (e.g. when using code generators) a small table gives an overview, which method does not need to fulfil the relating rule.
6) **MISRA**: Reference to MISRA-C: 2004 coding conventions that are directly or indirectly covered by the SCS rule.
7) **Example**: Where the rule is not self-explanatory, and in order to help the reader to understand the rule, an **incorrect** code example and a **correct** code example is given which is specific to the rule it applies to.

6.3 Description Format of a Guideline

Each guideline has a title and five description sections:

1) **ID**: The unique ID of the coding convention. These assigned IDs will not be changed with future releases of this standard.
2) **Guideline**: The detailed requirement of the guideline and specifically, what is and what is not 'allowed'.
3) **Reason**: A brief explanation and the rationale behind why the guideline is important and its positive/negative effect(s) and/or impact.
4) **MISRA**: Reference to MISRA-C: 2004 coding conventions that are directly or indirectly covered by the SCS guideline.
5) **Example**: Where the guideline is not self-explanatory, and in order to help the reader to understand the guideline, an incorrect code example and a correct code example is given which is specific to the guideline it applies to.

6.4 Use and Definition of "required" and "advisory"

According to the modal verbs described in 6.5, general 'Importance levels' have been introduced. This applies only to the rules, not to the guidelines:

'**required**' - comply with the 'must'-rules.

'**advisory**' - comply with the 'should'-rules. These are requirements placed on the programmer that should normally be followed. However they do not have the mandatory status of 'required'-rules. Note that the status of 'advisory' does not mean that these items can be ignored, but that they should be followed as far as is reasonably practical. Formal deviations are not necessary for 'advisory'-rules, but may be raised if it is considered appropriate.

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

6.5 Use and Definition of the Words “must”, “should” and “may”

Where the following words are used in the description text of a coding convention (section *Rule* or *Guideline*) they have the following meaning (in decreasing level of importance).

For rules:

‘**must**’/'**shall**' – (with or without additional negation word ‘not’) for software ‘safety’, compliance (reference [DO-178B] / [DO-178C]), and Liebherr-Aerospace requirement reasons, the rule must be adhered to in the code. Any non-conformity with the rule will probably invalidate the software being used for avionics purposes at the DAL defined in the rule and therefore must be brought to the attention (problem report) of the Project Development Manager and Project QA Representative.

For guidelines:

‘**should**’ - is used as an internal (departmental ‘best practice’) recommendation.

For any other place in the coding convention description:

‘**may**’ - is used in explanation texts like ‘Reason’ and is not used to define any rule or guideline.

6.6 Compliancy Check

The compliancy of the source code to the coding conventions specified in this document can be checked by several methods. A detailed coverage matrix is provided in the [SVP]. The categories of method are briefly described below:

- **Automated Quality Check**: realised by static analysis tool(s). These tool(s) can analyse ‘C’ source code for specific coding ‘violations’ depending on the compiler being used, specific filters and specific ‘compiler personality files’. The results of the tool(s) can be error or warning messages.
- **Manual Review**: where an automated tool cannot be used with certainty, a manual review performed by software and test engineers must be held.

The initial step in maintaining (avionics compliant) software is to ensure observance of the conventions described in this document.

6.7 Annotating the Source Code

Static Code Analyzer Tools often allow source code annotations in order to support code-based suppressions of specific rule violations, or to indicate a specific allowed form not recognized by general check means but compliant to the SCS-Rule.

Source Code Annotations may be used in order to suppress the failure generation due to the reasons below:

- a rule cannot be followed by the developer: there is no technical solution to avoid the failure or deviation
- the deviation is a specific allowed form of a more general forbidden issue, but the check means are not precise enough to authorize automatically this specific form

Source code annotations must comply with following rules:

- Inside of a function, annotating multiple lines of source code with a single annotation is allowed. Annotations stretching over function scope are not allowed.

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

- Annotations are allowed within a class equivalent scope of the design (e.g. one annotation for an enum and another one for defines. Not allowed is one annotation for enum and defines)
- Unterminated annotations are not allowed (no usage of ++ indication for lines)

A comprehensive report of all source code annotations shall be produced and submitted to the project software accomplishment summary.

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7 Rules

7.1 Code Layout

7.1.1 Comments Style

ID: [LAT9-0201-00006]
[Type: Rule]

Rule: Source code **must** be commented to an extend that is necessary to establish the ability to read, understand, and maintain coude using only /* ... */ style comments.

DAL:

A	B	C	D
req	req	req	adv

Reason: The use of //.... style comments should be avoided, because this comment style is not part of the ISO-C standard from 1990 and may therefore not be interpreted identically by older compilers, particularly for nested comments and combined with #define.
For compatibility with C++, the //... comments are only part of the C language since 1999.
Complete, succinct, accurate, up-t-date, and formatted commentary improves the ability to read, understand, and maintain code.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 2.2

Example: wrong:

```
// setting Status to TRUE
bStatus_o = TRUE;
```

OK:

```
/* setting Status to TRUE */
bStatus_o = TRUE;
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.1.2 No Nested Comments

ID: [LAT9-0201-00124]
[Type: Rule]

Rule: Comments **must not** be nested.

DAL:

A	B	C	D
req	req	req	adv

Reason: Better documentation of the program and a simpler review of the code against the SW-design. Moreover, nested comments could be interpreted differently by the different compilers supporting this language extension to C-90.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 2.3

Example: wrong:

```
nTemp_o = nTemp_i;    /*      1. Comment
                              /* 2. Comment */
                      */
```

OK:

```
nTemp_o = nTemp_i;     /* 1. Comment */
                       /* 2. Comment */
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.1.3 “One-Statement-Block” Block Borders

ID: [LAT9-0201-00011]
[Type: Rule]

Rule: Even if a program block contains only one statement, block border brackets (braces) "{" and "}" **must be** inserted.

DAL:

A	B	C	D
req	req	req	adv

Reason: If changes to the program are made, often a statement is added to the "one-statement-block" without adding the needed block borders "{" and "}".

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 14.8, Rule 14.9

Example:

wrong:

```
if (nVar1 >= TEN_C)
    nVar2 = SIX_C;
    nVar1 = TWENTY_C;
...
```

OK:

```
if (a >= TEN_C)
{
    nVar2 = SIX_C;
}
nVar1 = TWENTY_C;
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.1.4 One Statement per Line

ID: [LAT9-0201-00012]
[Type: Rule]

Rule: Every statement other than “else if” **must be** written on a separate line.

DAL:

A	B	C	D
req	req	req	req

Reason: Readability is improved. Comments can be better placed in the source code.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: n/a

Example: wrong:

```
nServo = nIn * nInput; nOut = nFserv + TEN_C;
```

OK:

```
nServo = nIn * nInput;      /* calculate force demand */
nOut = nFserv + TEN_C;      /* add offset for output */
```

wrong:

```
Function1(nVar1 = nVar1 + TEN_C);
```

OK:

```
nVar1 = nVar1 + TEN_C;
Function1(nVar1);
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.1.5 Declaration of Variables on Separate Lines

ID: [LAT9-0201-00013]
[Type: Rule]

Rule: Every declaration of a variable **must be** written on a separate line.

DAL:

A	B	C	D
req	req	req	adv

Reason: This will improve readability.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: n/a

Example:

wrong:

```
...
sint32_t lAd_in; sint32_t lDa_out;
...
```

OK:

```
sint32_t lAd_In;    /* to read in from the A/D converter */
sint32_t lDa_Out;   /* to write to the D/A converter */
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.2 Environment

7.2.1 Floating-Point Implementations

ID: [LAT9-0201-00153]
[Type: Rule]

Rule: Floating point implementations must only be used when they are compliant to the [IEEE-754] standard.

DAL:

A	B	C	D
req	req	req	req

Reason: With floating-point arithmetic, a lot of problems can appear (e.g. precision, underflow, overflow, etc.). By using one recognized standard, the potential of problems is strongly limited.

Applicability:

SDDM1	SDDM2	SDDM3
yes	yes	yes

MISRA: Rule 1.5

Example: n/a

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.2.2 Programming According to ANSI C Definition

ID: [LAT9-0201-00014]
[Type: Rule]

Rule: To produce executable object code, only compilers compliant to the ANSI C90 standard language definition **must be** used (Ref: "ANSI C Standard" document in chapter 3).
Definition, usage and behavior of used language extensions supported by the compiler **must be** documented by the developer (*designer software*), including:

- interrupt
- asm
- inline functions (C99)
- boolean type (C99)
- long long type (C99)
- near, far, huge, …
- system
- trap
- …

DAL:

A	B	C	D
adv	adv	adv	adv

Reason: Programming according to this standard means that the portability of the program increases. The documentation of the used extensions facilitates also the portability as well as increases the degree of comprehensibility of potential consequences by the programmers.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 1.1

Example: n/a

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.2.3 Reliance in Undefined and Unspecified Behavior

ID: [LAT9-0201-00132]
[Type: Rule]

Rule: Implementation leading to undefined or unspecified behavior **must be** avoided.

DAL:

A	B	C	D
req	req	req	adv

Reason: Special source code statements or constructions not being documented in the ANSI-C or in the compiler documentation can produce unexpected results, specially by porting activities, but even when run on the same platform but under other circumstances.
For a list of examples of these issues see [ANSI-C90], Annexes G.1, G.2

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 1.2

Example: n/a

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.2.4 Implementation-Specific Behaviours

ID: [LAT9-0201-00152]
[Type: Rule]

Rule: Each usage of implementation-specific behaviour **must be** documented.

DAL:

A	B	C	D
req	req	req	adv

Reason: No reliance shall be given to implementation-specific behaviour which is not documented. Special source code statements or constructions not being documented in the ANSI-C can be used only if they are referenced in the compiler documentation.
Otherwise, the behaviour during execution can be indeterminate or can vary depending on compilation conditions.
Moreover, the documentation will be useful in case on porting the software on another platform.
For a list of examples of these issues see [ANSI-C90], Annex G.3

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 3.1

Example: n/a

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.2.5 Characters Sets

ID: [LAT9-0201-00015]
[Type: Rule]

Rule: Only the characters and escape sequences sets defined in the ISO-C standard, plus for comments in source code the characters “$“, “@“, and “`“, and their corresponding encoding defined here below **must be** used.

The allowed escape characters are the following:

```
\a, \b, \f, \n, \r, \t, \v, \’, \”, \?, \\ \0
```

The allowed ISO-C character subset and its encoding is specified in the table here below, plus the control characters representing horizontal tab, vertical tab, and form feed. This encoding is defined and documented in the [ISO 10646-1], in “ *ISO_2003_charts_0000_33FF_BasicLatin_CJKCompat ”*, in “*Table 2 – Row 00: Basic Latin ”*.

	HEX		HEX		HEX		HEX		HEX		HEX
Space	20	0	30			P	50			p	70
!	21	1	31	A	41	Q	51	a	61	q	71
“	22	2	32	B	42	R	52	b	62	r	72
#	23	3	33	C	43	S	53	c	63	s	73
		4	34	D	44	T	54	d	64	t	74
%	25	5	35	E	45	U	55	e	65	u	75
&	26	6	36	F	46	V	56	f	66	v	76
‘	27	7	37	G	47	W	57	g	67	w	77
(	28	8	38	H	48	X	58	h	68	x	78
)	29	9	39	I	49	Y	59	i	69	y	79
*	2A	:	3A	J	4A	Z	5A	j	6A	z	7A
+	2B	;	3B	K	4B	[	5B	k	6B	{	7B
,	2C	<	3C	L	4C	\	5C	l	6C	|	7C
-	2D	=	3D	M	4D	]	5D	m	6D	}	7D
.	2E	>	3E	N	4E	^	5E	n	6E	~	7E
/	2F	?	3F	O	4F	_	5F	o	6F		

In comments the following additional characters are allowed:

	HEX		HEX		HEX
$	24	@	40	`	60

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

Other characters than the following **must not be** used for filenames:

```
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
a b c d e f g h I j k l m n o p q r s t u v w x y z
0 1 2 3 4 5 6 7 8 9 _ .
```

DAL:

A	B	C	D
req	req	req	req

Reason: By not applying this rule, the maintainability and portability are not ensured, and unknown behaviours can happen.

- Section 5.2.1 of the ISO standard defines characters which are the minimum source character set for all compilers. These are the only characters that must be used, even if the compiler supports a larger character set, or supports alternative character sets.
- [ISO 10646-1] defines an international standard for mapping character sets to numeric values. Conforming to a subset of [ISO 10646-1] is necessary for maintainability and portability.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 3.2, Rule 4.1

Example: n/a

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3 Programming Techniques

These rules define the programming techniques that **must be** followed when writing code using the C programming language. Their use will reduce the probability of errors, improve programming style and increase testability of the code.

7.3.1 Naming Conventions

7.3.1.1 Filenames

ID: [LAT9-0201-00094]
[Type: Rule]

Rule: All filenames **must be** strictly unique (case insensitive + extension) within the entire (executable) project.

DAL:

A	B	C	D
req	req	req	adv

Reason: Due to file configuration management, linker need, and also to increase traceability and comprehensibility, the filenames must be unique in the same project.

For example, if two files have the exact same name and so if the '-I' option is changed, it is possible that the source will no longer build, or worst case will build but have silent semantic changes.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: n/a

Example: n/a

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.1.2 Overlapping Identifier Name of Outer Scope

ID: [LAT9-0201-00093]
[Type: Rule]

Rule: Identifiers in an inner scope **must not** use the same name as an identifier in an outer scope if they hide that outer identifier.

DAL:

A	B	C	D
req	req	req	adv

Reason: This will increase readability and prevent misunderstanding.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 5.2

Example:

wrong:

```
sint32_t lVar1;
:
{
    uint32_t lVar1;
    lVar1 = 1U;
}
```

OK:

```
sint32_t lVar1_Outer;
:
{
    uint32_t lVar1_Inner;
    lVar1_Inner = 1U;
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.1.3 Significance of Identifiers within the First 63 Chars

ID: [LAT9-0201-00098]
[Type: Rule]

Rule: To ensure that all the identifiers are always significant with any ANSI development environment, the length of each identifier **must not** exceed 63 characters.

The expression “identifier” includes:

- Functions / Operations and their parameters
- Variables: local, global
- Constants (const …)
- Defines / Macros / Macro Parameters (preprocessor symbols)
- Structures / Unions / Enumerations, and members of them
- Typedefs
- Labels (“case xxx :” of switch – cases)
- Tags

DAL:

A	B	C	D
req	req	req	req

Reason: This is one of the ANSI-C definitions that bring portability of the program to other compilers. Some development environments (e.g. compilers) do not support more than 63 characters significance. Each new development environment should be checked for their ability to cope with a significance of 63 characters.

This restriction must be adhered to in order to allow for portability to other compilers. Remark: The ISO standard actually requires external identifiers to be significant in the first 6 characters. However this is a severe restriction and it is accepted that for modern compilers/linkers at least 63 characters significance are supported.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: n/a

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

Example:

wrong:

```
#define AAABBBCCCDDDEEEFFFGGGHHHIIIJJJKKKLLLMMMNNNOOOPPPQQQRRRSSSTTTUUU_X_C 0x10

#define AAABBBCCCDDDEEEFFFGGGHHHIIIJJJKKKLLLMMMNNNOOOPPPQQQRRRSSSTTTUUU_Y_C 0x20
```

OK:

```
#define AAABBBCCCDDDEEEFFFGGGHHHIIIJJJKKKLLLMMMNNNOOOPPPQQQRRRSSS_X_C 0x10

#define AAABBBCCCDDDEEEFFFGGGHHHIIIJJJKKKLLLMMMNNNOOOPPPQQQRRRSSS_Y_C 0x20
```

wrong:

```
void Calculate_Servo ()
{
    sint32_t lAaaBbbCccDddEeeFffGggHhhIiiJjjKkkLllMmmNnnOooPppQqqRrrSssTttUuu_X;
    sint32_t lAaaBbbCccDddEeeFffGggHhhIiiJjjKkkLllMmmNnnOooPppQqqRrrSssTttUuu_Y;
    :
}
```

OK:

```
void Calculate_Servo ()

{

    sint32_t lAaaBbbCccDddEeeFffGggHhhIiiJjjKkkLllMmmNnnOooPppQqqRrrSssTtt_X;

    sint32_t lAaaBbbCccDddEeeFffGggHhhIiiJjjKkkLllMmmNnnOooPppQqqRrrSssTtt_Y;

    :

}
```

wrong:

```
void AnUnbelievableLongNameOfFunctionWhichIsNotCodingStandardCompliantWith81Characters ()

{

:

}

void AnUnbelievableLongNameOfFunctionWhichIsNotCodingStandardCompliantAsItHas85Characters ()

{

:

}

/* Both functions have the same first 63 characters */
```

OK:

```
void ANameOfFunctionWith31Characters()

{

:

}

void ANameOfFunctionWithExactly38Characters()

{

:

}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.2 Types, Declaration and Definition

7.3.2.1 Function Prototyping

ID: [LAT9-0201-00016]
[Type: Rule]

Rule: Functions **must** have prototype declarations which **must be** declared at file scope and be visible at both the function definition and call.

This could be done in an included file for functions required to be used by other files, or directly in the C-file where they are defined for functions required to be used only in this C-file.

DAL:

A	B	C	D
req	req	req	req

Reason: It is necessary to the compiler to know the function's return and parameters type before the function is used. Moreover, declaring functions at block scope may be confusing, and can lead to undefined behaviour.
The rule represents normal practice.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 8.1, Rule 8.6

Example: wrong:

```
--- c-file:
uint32_t FirstFunc(uint32_t lPara1_i, sint32_t lPara2_i)
{
    /* !!! Declaration within another function block */
    uint32_t SecondFunc(uint32_t lVar1_i, sint32_t lVar2_i);
    :
    /* Call of function without known its prototype */
    bRes = ThirdFunction(lPara1_i);
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

OK:

```
--- h-files or c-files:

uint32_t FirstFunc(uint32_t lPara1_i, sint32_t lPara2_i);
...
uint32_t SecondFunc(uint32_t lVar1_i, sint32_t lVar2_i);
...
bool_t ThirdFunc(uint32_t lVar1_i);
...
uint32_t FirstFunc(uint32_t lPara1_i, sint32_t lPara2_i)
{
    bRes = ThirdFunction(lPara1_i);
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.2.2 Macro Definition is Restricted

ID: [LAT9-0201-00021]
[Type: Rule]

Rule: The use of C macros (#define) **must be** restricted like specified in the following:

- the redefinition of the language syntax is not allowed
- the macro expansion-string is limited to a braced initialiser, a constant, a string literal, a parenthesized expression, a type qualifier, a storage class specifier, a block of instructions delimited with curly brackets, or comparison operators like defined in [LAT9-0201-00080]
- only balanced brackets of whatever type ( ), [ ], { } in the expansion-string are allowed
- no semicolon at the end of the expansion-string is allowed.

DAL:

A	B	C	D
req	req	req	adv

Reason: A #define is just an alias which will be replaced with the full expansion-string during the pre-processing phase of the compiler. Any other use could lead to unexpected behaviour when substitution is made, or to very hard-to-read code.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 19.4 (partly)

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

Example: Wrong:

```
/* non-parenthesized expression */
#define NULL ZERO_C
/* unbalanced () and language redefinition */
#define STARTIF if(

/* not in a block '{ }' construct */
#define ASSERT_C( bCondition )                          \
    sint8_t cInternal = (sint8_t)( bCondition );        \
    if ( ( (bool_t)( cInternal ) ) EQU_C FALSE )        \
    {                                                   \
        return FALSE;                                   \
    }                                                   \

/* not a type qualifier, use typedef instead */
#define float32_t float
```

OK:

```
/* Constant */
#define NULL 0
/* Parenthesised expression */
#define NULL (ZERO_C)

/* block construct for use of multiple statements */
#define ASSERT_C( bCondition )                                  \
{                                                               \
    sint8_t cInternal = (sint8_t)( bCondition );                \
    if ( ( (bool_t)( cInternal ) ) EQU_C FALSE )                \
    {                                                           \
        return FALSE;                                           \
    }                                                           \
}

/* Type qualifiers */
#define USES extern
#define RO const

/* Wrapping of comparison operators */
#define EQU_C ==
#define OR_C ||
#define AND_C &&
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.2.3 Undefine of Macro is Not Allowed

ID: [LAT9-0201-00128]
[Type: Rule]

Rule: *#undef* **must not** be used.

DAL:

A	B	C	D
req	req	req	req

Reason: The use of *#undef* can confuse readability and comprehension, especially through header includes.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 19.6

Example: wrong:

```
#undef PI
#define PI 3.14F
```

OK:

```
#define MY_PI 3.14F
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.2.4 Scope of Macro Definition

ID: [LAT9-0201-00129]
[Type: Rule]

Rule: Macros **must not** be *#define*'d within a block.

DAL:

A	B	C	D
req	req	req	req

Reason: Even if it is legal in C to place #define directives anywhere in a code file, placing them inside blocks is misleading as it implies that the scope is restricted to that block, which is not the case.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 19.5

Example: wrong:

```
uint32_t Modulo2(uint32_t lVar_i)
{
#define TWO_C 2
    uint32_t lRes;
    lRes = lVar_i % TWO_C;
    return (lRes);
}
```

OK:

```
#define TWO_C 2
:
uint32_t Modulo2(uint32_t lVar_i)
{
    uint32_t lRes;
    lRes = lVar_i % TWO_C;
    return (lRes);
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.2.5 Function-Like Macros in Handwritten Code

ID: [LAT9-0201-00130]
[Type: Rule]

Rule: The only ones function-like macros allowed are the ASSERT-macros "ASSERT" and "ASSERT_ERR" like defined here below. Their definition and usage **must** comply with the following restrictions:

- their valid expansion-strings (or 'macro body') are the ones presented in the example section below, applying the general macro definition rules (see [LAT9-0201-000Appendix 221], [LAT9-0201-00128], [LAT9-0201-00129]).
- their invocations are limited to perform validity checks of function parameters, meaning function arguments and used global variables.
- their invocations are placed as first statements in functions before any local variable definition or other statements
- the last ASSERT-macro invocation must be followed by a new block delimited with braces '{ }', where the effective function code is placed.
- they must be always invoked with all of their arguments
- arguments to them must not contain tokens that look like preprocessing directives

For all other case of function-like macro, a real function **must be** preferred.

DAL:

A	B	C	D
req	req	req	adv

Reason: Normally the use of function-like macros produces a less-robust code because of misuses.
As the use of these both function-like macros is strong restricted, the risk of misuse is avoided.

The use of these function-like macros provides a considerable enhancement of the code quality in term of readability, maintainability and comprehensibility. Due to them, the validity-checks of the function parameters are performed in a much more efficient way: no more confusing numerous nested if-else-if-else statements are needed, they simplify the source code by grouping the validity-check related statements at the top of the function and allow a properly and efficient exit of the function.

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	no

MISRA: Rule 19.7, Rule 19.8, Rule 19.9, Rule 19.10

Example: wrong: (definition)

```
/* Definition of not allowed function-like macro */
#define MODULO(lVar) (lVar % 2)
:
lVar2 = MODULO(lVar1) + 1;
```

wrong: (definition)

```
/* Does not follow the general macro definition rules */
#define ASSERT_C( bCondition )                    \
if ( ( (bool_t)( bCondition) ) EQU_C FALSE )      \
{                                                 \
    return FALSE;                                 \
};                                                \
```

OK: (definition)

Here is the valid definition of the macro ASSERT:

```
#define ASSERT_C( bCondition )                        \
{                                                     \
    sint8_t cInternal = (sint8_t)( bCondition );      \
    if ( ( (bool_t)( cInternal ) ) EQU_C FALSE )      \
    {                                                 \
        return FALSE;                                 \
    }                                                 \
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

OK: (definition)

Here is the valid definition of the macro ASSERT_ERR:

```
#define ASSERT_ERR_C( bCondition , AssError)          \
{                                                     \
    sint8_t cInternal = (sint8_t)( bCondition );      \
    if ( ( (bool_t)( cInternal ) ) EQU_C FALSE )      \
    {                                                 \
        This_iocp->Error = ( AssError );              \
        return FALSE;                                 \
    }                                                 \
}
```

Wrong: (usage)

```
bool_t DrvEeprom_Init (DrvEeprom_t * const This_iocp,
                       EepromDevice_t DeviceID_i)
{
    ASSERT_C (This_iocp NEQ_C NULL); /* OK */
    /* !!! Missing 2nd argument*/
    ASSERT_ERR_C ((DeviceID_i  != 0));
    /* !!! 2nd argument is a preprocessing directive */
    ASSERT_ERR_C ((DeviceID_i  < 10),
                  #pragma error "invalid parameter");

    /* !!! Missing block delimitation {} for function code */
    EepromDevice_t EepromID;
    EepromID = DeviceID_i;
    :
    /* !!! Placed after non-function-like macro statements */
    /* !!! Used to check other things than param. validity */
    ASSERT_C (This_iocp->DevID NEQ_C EepromID);
    :
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

OK: (usage)

```
bool_t DrvEeprom_Init (DrvEeprom_t * const This_iocp,
                       EepromDevice_t DeviceID_i)
{
    /* OK: all ASSERTs grouped at the top of the function */
    /* OK: no preprocessing directives  */
    /* OK: all arguments passed  */
    /* OK: only parameter validity check */
    ASSERT_C (This_iocp NEQ_C NULL);
    ASSERT_ERR_C ((DeviceID_i  != 0), ERR_PARAM);
    ASSERT_ERR_C ((DeviceID_i  < 10), ERR_PARAM);
    /* OK: new block begin, function code inside */
    {
        EepromDevice_t EepromID;
        EepromID = DeviceID_i;
        :
    }
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.2.6 Function-Like Macros in Generated Code

ID: [LAT9-0201-00157]
[Type: Rule]

Rule: Function-like macros **must** be invoked with all its arguments where each argument's data type is in accordance with the action performed by the macro. The arguments **must not** contain tokens that look like preprocessing directives. In the macro definition each instance of a parameter **must** be enclosed in parentheses.

DAL:

A	B	C	D
req	req	req	adv

Reason: Compilers perform fewer checks on function-like macros than on functions. The use of arguments whose data types are not according to their usage can result in dangerous and unpredictable behavior.

Applicability:

SDDM1	SDDM2	SDDM3
no	yes	yes

MISRA: Rule 19.8, Rule 19.9, Rule 19.10

Example: wrong:

```
#define add(x, y) ((x) + (y))

z = add (43, "45");        /* the second argument is a string */
```

OK:

```
#define add(x, y) ((x) + (y))

z = add (43, 45);          /* OK, z will be set to 88 */
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

wrong:

```
#define abs(x) ((x >= 0 ? x : -x)

z = abs ( a – b );
/* the part –x of abs results in –a – b instead of – (a – b) */
```

OK:

```
#define abs(x) (((x) >= 0 ? (x) : -(x))
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.2.7 Use of Numeric Constants

ID: [LAT9-0201-00046]
[Type: Rule]

Rule: If unsigned numeric constants are used, a "U" suffix **must** be applied

DAL:

A	B	C	D
req	req	req	req

Reason: Depending on the environment (16 Bit, 32 Bit), a constant may be interpreted as signed or unsigned. Also the "U"-suffix may be redundant for larger values, its presence improves the clarity of the code.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 10.6

Example: wrong:

```
#define AB_C 0x8000 /* signed int in 16-bit environment */
                    /* unsigned int in 32-bit environment */
```

OK:

```
#define AB_C (0x8000U) /* always interpreted as unsigned int*/
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.2.8 Octal Constants

ID: [LAT9-0201-00146]
[Type: Rule]

Rule: Octal constants (other than zero) **must not** be used.

DAL:

A	B	C	D
req	req	req	adv

Reason: Any integer constant beginning with a "0" (zero) is treated as octal. So there is a danger, for example, with writing fixed length constants.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 7.1

Example: wrong:

```
#define SPECIAL_MASK_C 07417      /* => 111 100 001 111 */
:
nMY_Value &= SPECIAL_MASK_C;
```

OK:

```
#define SPECIAL_MASK_C 0xF0F      /* => 1111 0000 1111 */
:
nMy_Value &= SPECIAL_MASK_C;
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.2.9 Basic Types

ID: [LAT9-0201-00047]
[Type: Rule]

Rule: The basic types of char, short, long and float **must not** be directly used: length-specific and sign-specified equivalents **must be** *typedef'd* (depending on the compiler and CPU) and used instead. A proposal is specified in the guideline [LAT9-0201-00150].
It is not necessary to use *typedef*s in the declaration of bit-fields.

DAL:

A	B	C	D
req	req	req	adv

Reason: The storage length of types can vary from compiler to compiler. It is safer if programmers work with types whose they know to be of a given length.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 6.1, Rule 6.3

Example: wrong:

```
signed char cVal1 = 1;
unsigned long lVal2 = 0xffffffff;
```

OK:

std_defs.h

```
typedef signed char     sint8_t;
typedef unsigned long   uint32_t;
```

c-file

```
#include "std_defs.h"
sint8_t  cVal1 = 1;
uint32_t lVal2 = 0xffffffff;
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.2.10 Usage of Static Variables

ID: [LAT9-0201-00048]
[Type: Rule]

Rule: *Static* local variables inside of a function **must not** be used.

DAL:

A	B	C	D
req	req	req	adv

Reason: Static local variable induces a 'memory effect' on local variables, similar to global variables, but they are not visible from outside the function. They could be considered quasi as 'Input' variables between calls to the same function at different times, but they are very difficult to test because not directly stimulatable from outside the function (no black-box test is possible).
It causes also 'side effects' in case the function is used in different contexts (reusability), and therefore can provoke non predictable reactions.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: n/a

Example: wrong:

```
void Func(void)
{
    static sint32_t lMy_Value = 0;
}
```

OK:

```
/* Global Definitions */
sint32_t lMy_Value_g;
void Func(void)
{
    lMy_Value_g = 0;
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.2.11 Array Declaration

ID: [LAT9-0201-00051]
[Type: Rule]

Rule: During its declaration, either the size of the array **must be** explicitly given or the array **must** be fully initialized. The array size **must** be positive.

DAL:

A	B	C	D
req	req	req	adv

Reason: Although it is possible to declare an array of incomplete type and access its elements, it is safer to do so when the size of the array may be explicitly determined.
Most compilers will detect a declaration without specified size, but they may fail if the array is declared as an external variable.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 8.12, Rule 1.2 (partly)

Example: wrong:

```
uint16_t nArray1_a[];
extern uint16_t nArray2_a[];
uint16_t nArray3_a[-10];
```

OK:

```
uint16_t nArray1_a[] = {VAL1_C, VAL2_C, VAL3_C};
extern uint16_t nArray2_a[SIZE_C];
uint16_t nArray3_a[10];
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.2.12 Array Initialisation

ID: [LAT9-0201-00052]
[Type: Rule]

Rule: Braces **must** be used to indicate and match the structure in the non-zero initialisation of arrays and structures.

DAL:

A	B	C	D
req	req	req	adv

Reason: This will increase readability and immunity against failures.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 9.2

Example: wrong:

```
sint32_t lArray_a[3][2] = { 1, 2, 3, 4, 5, 6 };
```

OK:

```
sint32_t lArray_a[3][2] = { { 1, 2 },
                            { 3, 4 },
                            { 5, 6 }};
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.2.13 Array Index Type

ID: [LAT9-0201-00111]
[Type: Rule]

Rule: The type of every array index **must be** of an integral type.

DAL:

A	B	C	D
req	req	req	adv

Reason: Memory overlaps may occur with uncontrolled operations using arrays. To simplify the coding and allow the adequate verification, the limit of the index must be clearly identifiable.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: n/a

Example:

wrong:

```
uint16_t nArray_a[MAX_C];
sint16_t nIdx;  /* Variable can be negative !! */
...
nArray_a[nIdx] = 0; /* What happens if nIdx < 0 ? */
```

OK:

```
uint16_t nArray_a[MAX_C];
uint16_t nIdx;
...
nArray_a[nIdx] = 0; /* OK, if nIdx < MAX_C */
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.2.14 Array Element Control Index

ID: [LAT9-0201-00053]
[Type: Rule]

Rule: The value of every array index **must be** clearly identifiable being within the bounds of the array size before accessing an array element.

It can be realized by the use of an unsigned integral constant index value (defines, enums, constants), or by checking that the variable index does not overlap the bound of the array. For this second mean, it must be identifiable at the function scope that the index does not overlap the array bound.

DAL:

A	B	C	D
req	req	req	adv

Reason: Memory overlaps may occur with uncontrolled operations using arrays.
It must be ensured that array indices are within the bounds of the array size before using them to index the array.
To allow the adequate verification, the limit of the index must be clearly identifiable.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 1.2 (partly)

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

Example:

wrong:

```
uint16_t nArray_a[MAX_C];

/* This loop will access more elements than the array owns! */
for (nIdx = 0; nIdx <= MAX_C; nIdx ++)
{
    nArray_a[nIdx] = 0;
}
```

wrong:

```
void MyFunction(uint16_t nVar1_i)
{
    uint16_t nIdx;
    uint16_t nArray_a[100];

    for (nIdx = 0; nIdx < nVar1_i; nIdx++)
    {
        nArray_a[nIdx] = 0;
    } /* Overlaps if nVar1_i > 100 */
}
```

OK:

```
uint16_t nArray_a[MAX_C];
for (nIdx = 0; nIdx < MAX_C; nIdx++)
{
    nArray_a[nIdx] = 0;
}; /* Size of the array is equal to the number of accessed
elements */
```

OK:

```
nDynamicalLimit = Foo();
if (nDynamicalLimit < MAX_C)
{
    for (nIdx = 0; nIdx < nDynamicalLimit; nIdx++)
    {
        nArray_a[nIdx] = 0;
    }
}  /* Index limit identifiable in function scope */
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

OK:

```
typedef enum axis_identifier_tag
{
    GearAxis_E = 0,
    NickAxis_E,
    RollAxis_E,
    KollectiveAxis_E,
    MaxAxis_E
}axis_identifier_t;

uint16_t nArray_a[MaxAxis_E];
:
nArray_a[GearAxis_E] = 0;
nArray_a[NickAxis_E] = 0;
nArray_a[RollAxis_E] = 0;
nArray_a[KollectiveAxis_E] = 0;
:
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.2.15 Implicit Type Conversions

ID: [LAT9-0201-00054]
[Type: Rule]

Rule: Implicit conversions between values from different base types, type widths or type signs **must not** be used. An explicit cast **must be** used instead.

DAL:

A	B	C	D
req	req	req	adv

Reason: The intention when restricting implicit conversion is to require that in a sequence of arithmetic operations within an expression, all operations should be conducted in exactly the same arithmetic type. Note that this does not imply that all operands in an expression are of the same type.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 10.1, Rule 10.2

Example: wrong:

```
uint32_t lValue;
sfloat32_t fValue;
sint8_t cChar;

lValue = fValue; /* Base types */
cChar = (fValue / TWO_C);  /* Base types, type widths */
cChar = lValue; /* type signs */
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

OK:

```
uint32_t lValue;
sfloat32_t fValue;
sint8_t cChar;

lValue = (uint32_t) fValue;
cChar = (sint8_t) (((sint32_t)fValue) / TWO_C);
cChar = (sint8_t) lValue;
```

wrong:

```
uint16_t nValue_1 = 1;
uint16_t nValue_2 = 3;
uint16_t nValue_3 = 65535;
uint16_t nValue_4 = 10;
sfloat32_t fDivide;
uint32_t lSum;

fDivide = nValue_1/nValue_2; /* incorrect = 0.0 */
fDivide = (sfloat32_t)(nValue_1/nValue_2); /* incorrect 0.0 */

lSum = nValue_3 + nValue_4; /* incorrect = 9 */
lSum = (uint32_t) (nValue_3 + nValue_4); /* incorrect = 9 */
```

OK:

```
uint16_t nValue_1 = 1;
uint16_t nValue_2 = 3;
uint16_t nValue_3 = 65535;
uint16_t nValue_4 = 10;
sfloat32_t fDivide;
uint32_t lSum;

fDivide = (sfloat32) nValue_1/nValue_2; /* correct = 0.333 */
fDivide = (sfloat32_t) nValue_1/(sfloat32_t) nValue_2)
/* correct = 0.333 */

lSum = (uint32_t) nValue_3 + nValue_4; /* correct = 65545 */
lSum = (uint32_t) nValue_3 + (uint32_t) nValue_4;
/* correct = 65545 */
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.2.16 Explicit Type Conversions with the "cast" Operator

ID: [LAT9-0201-00123]
[Type: Rule]

Rule: The value of a complex expression of *integer type* **must** only be cast to a type of the same signedness that is no wider than the underlying type of the expression.

In the same way, the value of a complex expression of *floating type* **must** only be cast to a floating type that is narrower or of the same size.

If values are cast from bigger to shorter types a comment must be included explaining what bits and information is lost and what functional impact this loss has.

A possible solution can be to split the complex expression by means of temporary variables and extra statements, or to cast the different values separately.

Remark: Casting of float values to integer types only truncates and does not "round" the value.

DAL:

A	B	C	D
req	req	adv	adv

Reason: Conversions on complex expressions are often a source of confusion and it is therefore requested to be careful.
In case the cast is from bigger to smaller type it is necessary to explain not only that a loss of data occurs but also the functional impact of a potential loss of data. This allows other developers using this function to implement correct reaction on this behavior in calling functions and code reviewers to decide if the implemented behavior in case of data loss is compliant with the low-level requirements and overall intended behavior of the software.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	no

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

MISRA: Rule 10.3, Rule 10.4

Example:

```
uint32_t lValue_1;
uint16_t nValue_2;
uint16_t nValue_3;
float32_t fValue_4;
float32_t fValue_5;
float64_t dValue_6;
uint32_t lValue_7 ;
```

wrong:

```
lValue_1 = (uint32_t)(nValue_2 + nValue_3);
fValue_4 = (float32_t)(nValue_2 + nValue3);
nValue_2 = (uint16_t)(fValue_4 + fValue_5);
dValue_6 = (float64_t)(fValue_4 / fValue_5);
lValue_1 = (uint32_t)(nValue_2 * nValue_3);
```

OK:

```
lValue_1 = (uint32_t)nValue_2 + (uint32_t)nValue_3;
fValue_4 = (float32_t)nValue_2 + (float32_t)nValue3;
nValue_2 = (uint16_t)fValue_4 + (uint16_t)fValue_5;
dValue_6 = (float64_t)fValue_4 / fValue_5;
lValue_1 = (uint32_t)nValue_2 * nValue_3;

nValue_2 = (uint16_t)(lValue_1 * lValue_7); /* ! */
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.2.17 Casting of Pointers

ID: [LAT9-0201-00055]
[Type: Rule]

Rule: Type casting from any type to or from pointers **must not** be used, except if the target element can hold the whole information and qualification of the source element of the conversion.

This definition implies that:
- A function pointer can be converted to or from an integral type only
- An object pointer can be converted to or from:
  - an integral type
  - another pointer to object type
  - a pointer to void
- The conversion does not truncate the expected information
- The conversion does not provoke an invalid alignment
- The conversion does not remove any *const* or *volatile* qualification from the type addressed by a pointer

DAL:

A	B	C	D
req	req	adv	adv

Reason: Casting of pointers can lead to undefined or implementation-defined behaviour, and also circumvents type integrity.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 11.1, Rule 11.2, Rule 11.3, Rule 11.4, Rule 11.5

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

Example: wrong:

```
uint16_t nShort;
uint32_t lLong;
uint32_t *lPointerToLong_p;
MyStruct_t *PointerToStruct;
uint8_t  *Ptr8;
uint32_t *Ptr32;

lPointer_To_Long_p = &lLong; /* (OK) */

nShort = (uint16_t)lPointer_To_Long_p; /* Little/Big Endian? */
nShort = (uint16_t *)lPointer_To_Long_p; /* Loss of inform. */

PointerToStruct = &AFunction; /*What is the content of Struct?*/

Ptr32 = (uint32_t *)Ptr8; /* Probably incompatible alignment */
```

OK:

```
uint32_t lLong1;
uint32_t lLong2;
uint32_t *lPointerToLong_p;

lLong2 = (uint32_t *)lPointerToLong_p;
lPointerToLong_p = &lLong1;
lLong2 = *lPointerToLong_p;
```

OK:

```
#define SFR_BASE_ADDR_C 0x00000100UL
SFR_t *SFR_p = (SFR_t *)(SFR_BASE_ADDR_C);
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.2.18 Variable and Pointer Initialization Usage

ID: [LAT9-0201-00056]
[Type: Rule]

Rule: All declared variables and pointers used in a module **must** be initialized before being used.

DAL:

A	B	C	D
req	req	req	req

Reason: Uninitialized variables/pointers can point to spurious memory locations that have unknown data content. Consequently any data read from these locations will be completely random.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 9.1

Example: wrong:

```
uint32_t Func(void)
{
    uint32_t lResult;
    uint32_t lValue;
    uint32_t *lIn1_p;
    uint32_t lSpeed;

    lSpeed = *lIn1_p;
    lResult = lValue + 1;
    return lResult;
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

OK:

```
#define AD_SPEED_ADR_C 0xABCDEFU
uint32_t Func(void)
{
    uint32_t lResult = 0;
    uint32_t lValue = 0;
    uint32_t *lIn1_p;
    uint32_t lSpeed;

    lIn1_p = AD_SPEED_ADR_C;
    lSpeed = *lIn1_p;
    lResult = lValue + 1;
    return lResult;
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.2.19 Complexity of Pointer Constructions Limited to 2

ID: [LAT9-0201-00058]
[Type: Rule]

Rule: The indirection depth inside an instruction **must** be limited to 2.

DAL:

A	B	C	D
req	req	req	adv

Reason: The more the indirection is high, the more a strong abstraction is needed to understand the expression. A depth of 2 allows to use a single pointer as function output parameter: an output parameter passed per value induces an additional indirection depth. For more complex construction, intermediates types and variables with pointers are needed. Therefore this limitation does not preclude any software possibility.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 17.5

Example: wrong:

```
sint32_t ***lPointer_p;

:
***lPointer_p = 0;
:
```

OK:

```
sint32_t *lPointer1_p;
sint32_t **lPointer2_p ;
uint32_t lIn = 0;
:
*lPointer2_p = &lIn;
lPointer1_p = *lPointer2_p;
*lPointer1_p = 0;
:
**lPointer2_p = 0xFEED;
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.2.20 Explicit Type for Declaration of Variables and Functions

ID: [LAT9-0201-00063]
[Type: Rule]

Rule: Every variable or function declaration **must** have explicit types.

DAL:

A	B	C	D
req	req	req	adv

Reason: When no explicit type is specified, the hardware dependant int type is implicitly used.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 8.2

Example: wrong:

```
DoSomething();
extern nVar1_g;
const nVar2_c;
```

OK:

```
void DoSomething(void);
extern uint16_t nVar1_g;
const uint16_t nVar2_c;
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.2.21 Enum Initialization

ID: [LAT9-0201-00066]
[Type: Rule]

Rule: In an enumerator list, the "=" construct **must not** be used to explicitly initialize member other than the first, unless all items are explicitly initialized.

DAL:

A	B	C	D
req	req	req	adv

Reason: Explicit initialisation of all items in the list prevents the mixing of automatic and manual allocation, which is error prone. So only all, the first one, or none of the elements should be initialized explicit.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 9.3

Example:

wrong:

```
enum my_enum_e
{
    Value1_E,
    Value2_E = 4,
    Value3_E
};
```

OK:

```
enum my_enum1_e
{
    Value1_E = 0,
    Value2_E = 4,
    Value3_E = 5
};
```

OK:

```
enum my_enum2_e
{
    Var1_E = 0,
    Var2_E,
    Var3_E
};
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.2.22 Bitfields Types

ID: [LAT9-0201-00149]
[Type: Rule]

Rule: Bit fields **must** only be defined to be of type *unsigned int* or *signed int* with the restriction that a signed bit field **must** be at least 2 bits long.

To help covering the first part of this rule, it is suggested to use specific *typedefs* for bitfields types like specified in the guideline [LAT9-0201-00150].

DAL:

A	B	C	D
req	req	req	adv

Reason: The behavior is not defined in the C-Standard for other types than *signed* or *unsigned int* like just '*int*', *enum*, *short*, *char* etc. Therefore the interpretation is C-compiler dependent and not portable because unpredictable.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 6.4, Rule 6.5

Example: wrong:

```
typedef struct adc_error_bit_tag
{
   signed char    StdError:4;          /* 'char' not allowed */
   signed int     HspFrequencyError:1; /* signed int < 2 bits */
} adc_error_bit_t;
```

OK:

```
typedef signed int sbitfield_t;     /* Suggestion */
typedef unsigned int ubitfield_t;   /* Suggestion */
:
typedef struct adc_error_bit_tag
{
   sbitfield_t    StdError:4;
   ubitfield_t    HspFrequencyError:1;
} adc_error_bit_t;
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3 Statements

7.3.3.1 Dead Code

ID: [LAT9-0201-00019]
[Type: Rule]

Rule: There **must not** be any dead code.

'Dead code' means branches of code which exist as a result of a software development error but cannot be executed in any operational configuration. 'Dead code' must not be confused with 'Deactivated code'.

DAL:

A	B	C	D
req	req	req	adv

Reason: Dead code is not executable and not traceable to a software or system requirement. Therefore program size increases unnecessarily. By avoiding dead code, the software becomes more reliable and easier to read, to test and to maintain.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 14.1

Example: wrong:

```
sint32_t Function_xyz()
{
    uint32_t lValue1;
    :
    if (lValue1 < 0) /* Always false because value always >0*/
    {
    :                /* dead code */
    }
    return lResult;
    lResult = 1;     /* dead code */
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3.2 Conditions with Invariant Result

ID: [LAT9-0201-00138]
[Type: Rule]

Rule: Conditions with invariant results producing dead code **must not** be used. A condition, that is always true, only produces redundant code. If the condition is always false, this will also lead to dead code.

DAL:

A	B	C	D
req	req	req	adv

Reason: Dead code is not executable and not traceable to a software or system requirement. Therefore program size increases unnecessarily. By avoiding dead code, the software becomes more reliable and easier to read, to test and to maintain.

Applicability:

SDDM1	SDDM2	SDDM3
yes	yes	yes

MISRA: Rule 13.7

Example: wrong:

```
sint32_t Function_xyz()
{
    uint32_t lValue1;
    :
    if (lValue1 < 0) /* Always false because value always >0*/
    {
    :                /* dead code */
    }
    return lResult;
    lResult = 1;     /* dead code */
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3.3 Trigraphs

ID: [LAT9-0201-00020]
[Type: Rule]

Rule: Trigraphs **must not** be used.

DAL:

A	B	C	D
req	req	req	req

Reason: Trigraphs are denoted by a sequence of 2 question marks followed by a specified third character. They can cause accidental confusion with other uses of two question marks.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 4.2

Example: wrong:

```
uint8_t cTilde[] = "??-";     /* trigraph "??-" is "~" */
```

OK:

```
uint8_t cTilde[] = "~";
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3.4 Use of "goto" and Labels

ID: [LAT9-0201-00032]
[Type: Rule]

Rule: "goto" statements and labels other than for switch-case constructs **must not** be used.

DAL:

A	B	C	D
req	req	req	adv

Reason: Unconditional jumps destroy the program flow and are difficult to review.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 14.4

Example:

wrong:

```
while (i < nMax)
{
    if (Func(i) NEQ_C 0)
    goto error;
    i++;
}
goto no_error;
error:
    {
        nErrorCount++;
    }
no_error:
    :
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

OK:

```
nErr = 0;
while ((nErr EQU_C 0) AND_C (i < TEN_C))
{
    nErr = Func(i);
    i++;
}
if (nErr NEQ_C 0)
{
    nErrorCount++;
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3.5 Embedding Assembler Code into C Code

ID: [LAT9-0201-00126]
[Type: Rule]

Rule: Assembly language **must be** encapsulated and isolated from normal C code. This can be done by using either functions completely written in assembler, or C functions or C inline-functions, both consisting of inline assembly instructions only. For use of C inline-functions, see restrictions in [LAT9-0201-00014] and [LAT9-0201-00074].

DAL:

A	B	C	D
req	req	adv	adv

Reason: This will increase the portability of the code and avoid unwished side effects of inline-assembler code on C-code (e.g. access to registers which are also used by the compiler).

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 2.1

Example: wrong:

```
void MyCFunc(sint32 nPar_i)
{
    :
    nVar = nPar_i;
    asm("nop");
    OtherCFunc(nVar);
    :

    if (nVar > 1)
    {
        asm("LRLK AR0,0C00h");
        asm("LRLK AR1,0800h");
        ...
    }
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

OK:

```
inline void NOP()
{
    asm("   nop");
}
void FunctionWithInlineAsm(void)
{
    asm("LRLK AR0,0C00h");
    asm("LRLK AR1,0800h");
    ...
}
:
void MyCFunc(sint32 nPar_i)
{
    :
    nVar = nPar_i;
    NOP();
    OtherCFunc(nVar);
    :
    if (nVar > 1)
    {
        FunctionWithInlineAsm();
    }
}
```

OK: (Note: this variant is compiler dependent (keyword ‘asm’))

```
asm void AsmFunc(void)
{
    LRLK AR0,0C00h
    LRLK AR1,0800h
    ...
}
void MyOtherCFunc(sint32 nPar_i)
{
    :
    if (nPar_i > 1)
    {
        AsmFunc();
    }
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3.6 Logical Expressions Enclosed in Brackets

ID: [LAT9-0201-00034]
[Type: Rule]

Rule: Each logical expression **must be** enclosed in brackets.

DAL:

A	B	C	D
req	req	req	req

Reason: Makes code easier to read and review; ambiguity of the expressions is prevented; code is more reliable and subsequently “safer”.

Applicability:

SDDM1	SDDM2	SDDM3
yes	yes	yes

MISRA: n/a

Example:

wrong:

```
if (nIn EQU_C 0 AND_C nJn >= FIVE_C)
{
    :
}
```

OK:

```
if ((nIn EQU_C 0) AND_C (nJn >= FIVE_C))
{
    :
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3.7 Use Restriction of Assignment Statement

ID: [LAT9-0201-00036]
[Type: Rule]

Rule: An assignment statement **must not** be used as a condition expression.

DAL:

A	B	C	D
req	req	req	adv

Reason: Code is ambiguous and not explicit enough. It could be misunderstood as "==". It is recommended to dissociate the assignment and the "if" statement.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: n/a

Example:

wrong:

```
if ((nXn = nYn) > 0)
    :
```

OK:

```
nXn = nYn;
if (nXn > 0)
    :
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3.8 Test against Zero

ID: [LAT9-0201-00037]
[Type: Rule]

Rule: Test of a value against zero **must** be made explicit.

DAL:

A	B	C	D
req	req	req	adv

Reason: This rule is in the interests of clarity, and makes clear the distinction between integers and logical values.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 13.2

Example: wrong:

```
sint32_t lXn;
if (lXn)
```

OK:

```
sint32_t lXn;
if (lXn NEQ_C 0)
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3.9 "switch" Statement Scope for "case" and "default"

ID: [LAT9-0201-00038]
[Type: Rule]

Rule: A switch statement **must be** built with all *case* labels and the *default* label at its direct closely-enclosing scope.

DAL:

A	B	C	D
req	req	req	req

Reason: A *switch* structure becomes difficult to read and to test if clause labels are used at different scope.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 15.1

Example: wrong:

```
switch (nPosition)
{
    case CASE1_C:
        nCommand = nPosition + TEN_C;
        if (nCommand > POSITION_MAX_C)
        {
            case CASE2_C:
                nCommand = nPosition - TEN_C;
                break;
        }
        break;
    default:
        nCommand = INVALID_C;
        break;
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

OK:

```
switch (nPosition)
{
    case CASE1_C:
        nCommand = nPosition + TEN_C;
        break;
    case CASE2_C:
        if (nCommand > POSITION_MAX_C)
        {
            nCommand = nPosition - TEN_C;
        }
        break;
    default:
        nCommand = INVALID_C;
        break;
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3.10 Use of “break” in “switch / case” Branches

ID: [LAT9-0201-00115]
[Type: Rule]

Rule: Each non-empty *case*-branch **must** end with a "break" instruction.

DAL:

A	B	C	D
req	req	req	req

Reason: The use of "break" is needed to separate properly the execution of the clause-branches. Executing more than one clause-branch must be avoided.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 15.2

Example: wrong:

```
switch (nPosition)
{
    case CASE1_C:
        nCommand = nPosition + TEN_C;
    case CASE2_C:
        nCommand = nPosition - TEN_C;
...
```

OK:

```
switch (nPosition)
{
    case CASE1_C:
        nCommand = nPosition + TEN_C;
        break;
    case CASE2_C:
        nCommand = nPosition - TEN_C;
        break;
...
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3.11 Minimal "switch" Statement

ID: [LAT9-0201-00114]
[Type: Rule]

Rule: Every "switch" statements **must** contain at least one *case* clause and exactly one *default* clause placed as final clause.

DAL:

A	B	C	D
req	req	req	req

Reason: Without *case*, a *switch* does not make sense. The *default* branch must be used to detect an error if no other *case* matches the *switch* value.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 15.3, Rule 15.5

Example: wrong:

```
switch (nPosition)
{
    case CASE1_C:
...
}
```

OK:

```
switch (nPosition)
{
    case CASE1_C:
...
    default:
        nCommand = INVALID_C;
        break;
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3.12 "switch" Expression as Boolean Value

ID: [LAT9-0201-00116]
[Type: Rule]

Rule: A *switch* expression **must not** represent a Boolean value.

DAL:

A	B	C	D
adv	adv	adv	adv

Reason: Boolean is not adequate for a Switch/Case, using an *if-else* condition is the right way.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 15.4

Example: wrong:

```
switch (nPosition EQU_C nValue)
...
```

OK:

```
if (nPosition EQU_C nValue)
...
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3.13 Usage and Structure of "if-else if"-Conditions

ID: [LAT9-0201-00039]
[Type: Rule]

Rule: All *if - else if* constructs **must** contain a final else clause.

DAL:

A	B	C	D
adv	adv	adv	adv

Reason: The requirement for a final *else* statement is defensive programming. The *else* statement should either take appropriate action or contain a suitable comment why no action is taken. This should at least report an "unrecognised condition" error.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	no

MISRA: Rule 14.10

Example: wrong:

```
if (...)
{
    ...
}
else if (...)
{
    ...
}
```

OK:

```
if (...)
{
    ...
}
else if (...)
{
    ...
}
else
{
    ...
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3.14 Use of "for" Loop Counter

ID: [LAT9-0201-00040]
[Type: Rule]

Rule: The loop counter **must** be a local variable and has to be initialized in the *for* construct - where there is no initialization, it is recommended to use *while*.

DAL:

A	B	C	D
req	req	req	adv

Reason: Loops must be implemented in such a way that every programmer/reviewer easily and clearly understands the nature and function of the loop.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: n/a

Example: wrong:

```
sint32_t i = TEN_C;
for (i; i > 0; i--)
{
:
}
```

OK:

```
for (i = TEN_C; i > 0; i--)
{
:
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3.15 "for" Loop Must Have an End Condition

ID: [LAT9-0201-00117]
[Type: Rule]

Rule: Each *for* loop **must** have an end condition. Never ending loops **must** be implemented with the *while (TRUE)* construct.

DAL:

A	B	C	D
req	req	req	adv

Reason: Loops must be implemented in such a way that every programmer/reviewer easily and clearly understands the nature and function of the loop.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: n/a

Example: wrong:

```
for (i = 1; ; i++)
{
:
}
```

OK:

```
for (i = 1; i < TEN_C; i++)
{
:
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3.16 Variable Used as End Border in "for" Loop

ID: [LAT9-0201-00119]
[Type: Rule]

Rule: If the end border of a *for*-loop is a variable, then it **must be** recognizable at the function scope that the loop-index will not provoke an undefined behavior like an array-overflow, a division by zero, etc.

DAL:

A	B	C	D
adv	adv	adv	adv

Reason: This is necessary to minimize the risk of overflows through a better control of data ranges. Loops must be implemented in such a way that every programmer/reviewer easily and clearly understands the nature, function and consequences of the loops.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: n/a

Example:

```
uint8_t cMyArray_a[ARRAY_SIZE_C];
```

wrong:

```
void InitArray(uint16_t nNbOfVals_i)
{
   uint16_t i;

   for (i = 0; i < nNbOfVals_i; i++)
   {
      cMyArray_a[i] = ARRAY_SIZE_C - i;
   }
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

OK:

```
void InitArray(uint16_t nNbOfVals_i)
{
   uint16_t i;

   for (i = 0; i < nNbOfVals_i, nNbOfVals_i < ARRAY_SIZE_C; i++)
   {
      cMyArray_a[i] = ARRAY_SIZE_C - i;
   }
}
```

OK:

```
void InitArray(uint16_t nNbOfVals_i)
{
   ASSERT_C(nNbOfVals_i < ARRAY_SIZE_C);
   {
      uint16_t i;

      for (i = 0; i < nNbOfVals_i; i++)
      {
         cMyArray_a[i] = ARRAY_SIZE_C - i;
      }
   }
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3.17 Expressions Used in "for" Loop Statements

ID: [LAT9-0201-00120]
[Type: Rule]

Rule: Only expressions concerned with loop control **must** appear within a *for* statement.

DAL:

A	B	C	D
req	req	req	adv

Reason: Loops must be implemented in such a way that every programmer/reviewer easily and clearly understands the nature and function of the loop.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 13.5

Example: wrong:

```
for (i = 0, lVar = 10; i < lMax; i++, lVar1++) /* lVar ? */
{
    DoSomething(i);
}
```

OK:

```
for (i = 0; i < TEN_C; i++)
{
    DoSomething(i);
}
```

OK:

```
for (i = 0; (i < lMax) && (lMax < TEN_C) || bOK != TRUE; i++)
{
    bOK = DoSomething(i);
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3.18 Change of Loop Counter of "for" Loop

ID: [LAT9-0201-00121]
[Type: Rule]

Rule: Loop counter variables **must not** be changed within the loop body.

DAL:

A	B	C	D
req	req	req	adv

Reason: Loops must be implemented in such a way that every programmer/reviewer easily and clearly understands the nature and function of the loop.
Modifying the loop variable outside the loop construct could lead to unreliable code.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 13.6

Example: wrong:

```
for (i = 0; i < lMax; i++)
{
:
    i++;
:
}
```

OK:

```
for (i = 0; i < TEN_C; i++)
{
:
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3.19 No Floating Point Type for Counter of "for" Loop

ID: [LAT9-0201-00122]
[Type: Rule]

Rule: Floating point variables **must not** be used for loop counters.

DAL:

A	B	C	D
req	req	req	req

Reason: Loops must be implemented in such a way that every programmer/reviewer easily and clearly understands the nature and function of the loop.
Using floating point variables for loop counter could lead to unreliable code.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 13.4

Example: wrong:

```
sfloat32_t fCntr;
for (fCntr = 0; fCntr < 10.0; fCntr++)
{
:
}
```

OK:

```
sint32_t i;
for (i = 0; i < TEN_C; i++)
{
:
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3.20 Usage of Break and Continue Statements in Iterative Structures

ID: [LAT9-0201-00041]
[Type: Rule]

Rule: *Continue* statements in *for, do*, and *while* statements **must not** be used. There **must** be at most one *break* statement.

DAL:

A	B	C	D
req	req	req	adv

Reason: Similar to *goto*. These statements de-structure the code. By eliminating them, code is easier to understand.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 14.5, Rule 14.6

Example:

wrong:

```
for (i = 0; i < nMax; i++)
{
    if (nArray_a[i] < 0)
    {
        /* skip negative elements*/
        continue;

        /* work on with positive elements */
        nError = Func_xyz(i);
        if (nError EQU_C 0)
        {
            break;
        }
    }
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

OK:

```
for (i = 0; i < nMax; i++)
{
    if (nArray_a[i] < 0)
    {
        /* skip negative elements*/
    }
    else
    {
        /* work on with positive elements */
        nError = Func_xyz(i);
        if (nError EQU_C 0)
        {
            break;
        }
    }
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3.21 Only One Side Effect or Control Flow Change per Non-Null Statement

ID: [LAT9-0201-00042]
[Type: Rule]

Rule: Each non-null statement **must** contain exactly *one side effect or one function call*. This covers also the case where function calls are performed in function parameters.

DAL:

A	B	C	D
req	req	req	req

Reason: The result of an expression must be exploitable and identical under any order of evaluation that the ANSI-C standard permits.
Multiple assignments or/and function calls in one statement are complex to test entirely, and the ANSI-C does not specify the order of evaluation.
Consequently, in case of interdependence, the evaluation order of multiple assignments in one statement is compiler depended: there is no guarantee that the same result will occur when using different compilers.
The coding of: operators increment/decrement, function arguments, functions pointers, function calls, assignment statements are impacted by this rule in order to avoid any undefined and unspecified behaviour.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 12.2, Rule 12.13, Rule 14.2

Example: Non-null statement:

wrong:

```
nCtr + 1; /* no usable data flow and no control flow */
```

OK:

```
nCtr++;
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

Multiple side effects on the same object:

wrong:

```
nVar1 = nVar2 * nVar1++;  /* Order of evaluation unspecified */
```

OK:

```
nVar1 = nVar2 * nVar1;
nVar1++;
```

Increment or decrement operators:

wrong:

```
nCtr = TWO_C;
MyArray_a[nCtr] = ++nCtr + 1; /* Order evaluation unspecified */
      /* => MyArray_a[2] = 3 + 1 or MyArray_a[3] = 3 + 1 ?? */
```

OK:

```
nCtr = TWO_C;
nCtr++;                    /* nCtr = 3 is guaranteed */
MyArray_a[nCtr] = nCtr + 1;  /* MyArray_a[3] = 3 + 1 */
```

Function Arguments:

wrong:

```
nVar1 = MyFct(nPar, nPar++); /* Evaluation order unspecified */
```

OK:

```
nPar2 = nPar1 + 1;
nVar1 = MyFct(nPar1, nPar2);
```

wrong:

```
nVar1 = MyFct(nPar, nPar + nValue);  /* Not easy testable */
```

OK:

```
nPar2 = nPar1 + nValue; /* Intermediate value: testable */
nVar1 = MyFct(nPar1, nPar2);
```

Function Call as Argument (nested function call):

wrong:

```
    nVar1 = Fct1(nVar2, Fct2(nVar3));
        /* Return value Fct2() not testable */
```

OK:

```
    nResult = Fct2(nVar3);
    nVar1 = Fct1(nVar2, nResult)
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

Function Pointers:

wrong:

```
Ptr_pfn->StartTask(Ptr_pfn++);

/* Dependence and evaluation order unspecified in ANSI-C */
```

OK:

```
Ptr2_pfn = Ptr1_pfn + 1;

Ptr1_pfn->StartTask(Ptr2_pfn);

Ptr1_pfn++;
```

Function Calls:

wrong:

```
nRes = Fct1(nVal1) + Fct2(nVal2);

/* Evaluation order unspecified => maybe Fct2 called first? */

/* Potential side effect: does Fct2() set
   a global variable also used in Fct1() ? */
```

OK:

```
nRes = Fct1(nVal1);

nRes = nRes + Fct2(nVal2);
```

Multiple / Nested assignments:

wrong:

```
nVar1 = nVar2 = nVar3; /* Order of evaluation unspecified */
```

OK:

```
nVar1 = nVar3;
nVar2 = nVar3;
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

Multiple volatile objects:

```
volatile uint16_t  *nVar1_p;
volatile uint16_t  *nVar2_p;
```

wrong:

```
*nVar2_p = *nVar1_p;
```

OK:

```
uint16_t nVar3;

nVar3 = *nVar1_p;
*nVar2 = nVar3;
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3.22 NULL Statements

ID: [LAT9-0201-00140]
[Type: Rule]

Rule: A null statement **must** only occur on a line by itself, and **must not** have any other text on the same line.

DAL:

A	B	C	D
req	req	req	req

Reason: Following this rule enables a static checking tool to warn of null statements appearing on a line with other text, which would normally indicate a programming error.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 14.3

Example: wrong:

```
#define BLANK_C
...
  BLANK_C; /* comment */
```

OK:

```
#define BLANK_C
...
  /* comment */
  BLANK_C;
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

wrong:

```
...
while (...)
{
  /* Comment1 */ ; /* Comment2 */
...
```

OK:

```
...
while (...)
{
  /* Comment1 */
  ;
  /* Comment2 */
...
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3.23 Pointer Arithmetic

ID: [LAT9-0201-00059]
[Type: Rule]

Rule: Pointer arithmetic **must not** be used unless it can be identified directly in the function that:
- the pointer points to an array to access the array elements
- the pointer object base type is the same as the one for the array elements
- the array boundary is visible and taken into account to perform calculations
- a subtraction of pointers operates with pointers addressing the same array

For any other case, the only allowed form of arithmetic by accessing values with indirection is the array indexing.

DAL:

A	B	C	D
req	req	adv	adv

Reason: By using pointers arithmetic, the risk is higher than by array indexing to perform accesses outside the array bound: the start address of an array can not be changed, while the pointer has to change. Moreover, the address calculated by pointer increment or decrement is dependent of the declared object size of the pointer, not of the currently pointed object size. This may be very confusing and is often a source of error.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 17.1, Rule 17.2, Rule 17.4

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

Example:

wrong:

```
uint32_t lVar1;
uint16_t *lPointer_p; /* Pointer object type size != array */

lPointer_p = (uint16_t *)&lVar1;  /* !!! */

lPointer_p++;    /* Where does point now the pointer ? */
lPointer_p += 1; /* Where does point now the pointer ? */

*lPointer_p = 0; /* Outside of the lVar1 memory cell... */
```

OK:

```
#define FIRST_ADDRESS_C 0x22222222
#define SECOND_ADDRESS_C 0x44444444

sint32_t lArray_a[100];
uint32_t *lPointer_p;   /* Same object type size as array */

lPointer_p = FIRST_ADDRESS_C;

lPointer_p = SECOND_ADDRESS_C;

lPointer_p = &lArray_a[5];
lPointer_p++;     /* Visibilty OK: 5 < 100, increment OK */
*lPointer_p = 0;
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3.24 Pointer Comparison

ID: [LAT9-0201-00060]
[Type: Rule]

Rule: Relational operators **must not** be applied to pointer types except if they are of the same type and point to the same array.

DAL:

A	B	C	D
req	req	req	req

Reason: Attempting to make comparison between pointers is fraught with difficulty, and in particular will produce undefined behavior if the two pointers do not point to the same object.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 17.3

Example:

```
sint32_t *lPointer1_p;
sfloat *fPointer2_p;
sint32_t lRegister1;
sfloat32_t fRegister2;
sint32_t *lPointer3_p;
sint32_t *lPointer4_p;
sint32_t lArray_a[100];
```

wrong:

```
lPointer1_p = &lRegister1;
fPointer2_p = &fRegister2;
if (lPointer1_p > fPointer2_p)
:
```

OK:

```
lPointer3_p = &lArray_a[5];
lPointer4_p = lArray_a;
if (lPointer3_p > lPointer4_p)
:
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3.25 Function Pointers

ID: [LAT9-0201-00061]
[Type: Rule]

Rule: Pointer to functions **must be** avoided if they are not statically coded (no dynamical initialization or change).

In any case, all functions pointed to by a single pointer to function **must be** identical in the number and type of parameters and in the return type.

DAL:

A	B	C	D
req	adv	adv	adv

Reason: The pointer to functions may result in unforeseeable results in case of an incorrect usage. All parameter passing checks and prototype checks may be disabled.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: n/a

Example:

```
typedef uint8_t (*TheFctPtrType_t) (uint32_t lPar1);
uint8_t MyFunction(uint32_t lPar1);
sint32_t lResult;
uint8_t cResult;
```

wrong:

```
sint32_t (*MyFctPtr_pfn)(sint16_t nPar1, sint32_t lPar2);
:
MyFctPtr_pfn = (*)(sint16_t, sint32_t)&MyFunction;
MyFctPtr_pfn = (*)()&MyFunction;
lResult = (*MyFctPtr_pfn)(0xDEAD, 0xBEEF);
```

OK:

```
const TheFctPtrType_t MyFctPtr_pfn = &MyFunction;
:
cResult = MyFctPtr_pfn(0xFEED);
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3.26 Structs and Unions Members

ID: [LAT9-0201-00064]
[Type: Rule]

Rule: All members of a structure (or union) **must** only be accessed via their name.

DAL:

A	B	C	D
req	req	req	adv

Reason: The memory representation of a structure (or union) is compiler-dependent, for example because of memory alignment problems. The intention of this rule is to avoid risks by accessing the members ‘by address’. It increases portability as well as code-readability.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: n/a

Example:

```
typedef struct MyStructType_tag
{
    uint8_t  cElt1;
    uint8_t  cElt2;
} MyStructType_t;
...
MyStructType_t MyStruct;
```

wrong:

```
uint8_t *ElementPtr_p;
ElementPtr = (uint8_t*)&MyStruct;
*ElementPtr++ = 1;
*ElementPtr++ = 2;
```

OK:

```
MyStruct.cElt1 = 1;
MyStruct.cElt2 = 2;
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3.27 Floating Point Comparisons

ID: [LAT9-0201-00067]
[Type: Rule]

Rule: Floating point comparisons == (EQU_C) and != (NEQ_C) **must not** be used.

DAL:

A	B	C	D
req	req	req	req

Reason: Due to rounding mechanisms inherent to floating point calculation, the complete value with its whole granularity is depending of its history of calculation. For example, the value fVal1 = (fVal1 / fVal2) could be lightly different if it is reverse calculated with fVal1 = fVal1 * fVal2.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 13.3

Example: wrong:

```
sfloat32_t fVal1;
sfloat32_t fVal2;

if (fVal1 EQU_C fVal2)
{
    ...
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

OK:

```
#define TOLERANCE_C 1E-14
sfloat32_t fVal1;
sfloat32_t fVal2;

fDiff = Absolut(fVal1 - fVal2);
if (fDiff <= TOLERANCE_C)
{
    ...
}

sfloat32_t Absolut(sfloat32_t fVar1)
{
    if (fVar1 < 0)
    {
        fVar1 = -fVar1;
    }
    return fVar1;
}
```

OK:

```
/* Prevention of division by zero */
if (fDivisor < (float32_t)0.0) || fDivisor > (float32_t)0.0)
{
    This_iocp->fValue =
     (float32_t)This_iocp->lRawValue / fDivisor;
}
else
{
    This_iocp->fValue = (float32_t)0.0;
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3.28 Floating Point Representation

ID: [LAT9-0201-00073]
[Type: Rule]

Rule: The underlying bit representation of floating point numbers **must not** be used in any way by the programmer.

DAL:

A	B	C	D
req	req	req	req

Reason: The exact storage layout used for floating numbers may vary from one compiler to another, and therefore no floating point manipulations should be made which rely on the way the numbers are stored. The in-built operators and functions, which hide the storage details from the programmer, should be used.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 1.2 (partly), Rule 12.12

Example: wrong:

```
float32_t fVar1;
float32_t *fPointer_p;

fPointer_p = &fVar1;

/* Deletes bit 5 */
*fPointer_p = *fPointer_p & (float32_t)0xFFFFFFEF;
/* The new value of fVar1 is unpredictable */
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

wrong:

```
/* Convert uint32 into float32 via 'union', e.g. for values
   transferred (e.g. via CAN) from another device don't using
   the same floating point representation, even with 32 bits */

typedef union CANMessage_tag
{
    float32_t fValueAsFloat;
    uint32_t  lValueAsInteger;
    :
} CANMessage_t;
:
float32_t fVar1;
CANMessage_t Message;
:
/* Read incoming CAN Message from Driver */
:
fVar1 = Message.fValueAsFloat;
/* Wrong: assumption on underlying bit representation */
```

OK:

```
fVar1 = (float32_t)Message.lValueAsInteger;
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3.29 Preprocessor Statement “#include”

ID: [LAT9-0201-00044]
[Type: Rule]

Rule: *#include* statements in a file **must** only be preceded by other preprocessor directives or comments, and each of *#include* directive **must** be followed by either a *<filename>* or *"filename"* sequence.

DAL:

A	B	C	D
req	req	req	adv

Reason: If files are not included at the head of a file, executable code before a *#include* directive may try to use items which would be defined in the header file.
There may be other forms to specify a filename, but to increase portability, only these two forms must be used.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 19.1, Rule 19.3

Example: wrong:

```
uint32_t lMyVar;   /* illegal: not a preprocessor statement */
:
#include glbdef.h  /* illegal: missing <> or "" */
```

OK:

```
#include "glbdef.h"
:
uint32_t lMyVar;
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3.30 Preprocessor Statement “defined”

ID: [LAT9-0201-00141]
[Type: Rule]

Rule: The defined preprocessor operator **must** only be used in one of the two standard forms:

```
defined ( identifier )
defined identifier
```

DAL:

A	B	C	D
req	req	req	req

Reason: Using other forms may produce undefined behavior.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 19.14

Example: wrong:

```
#if defined "nHallo"
```

OK:

```
#if defined nHallo
#if defined (nHallo)
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3.31 Preprocessor Operators # and

ID: [LAT9-0201-00142]
[Type: Rule]

Rule: There **must be** at most one occurrence of the # or ## preprocessor operators in a single macro definition. The two occurrences **must not** be mixed.

DAL:

A	B	C	D
req	req	req	adv

Reason: There is an issue of unspecified order of evaluation associated with the # and ## preprocessor operators.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 19.12, Rule 19.13

Example: wrong:

```
#define debug(s) printf("x" # s"=%d\n", x ## s)
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3.32 Preprocessor Directive with Identifiers

ID: [LAT9-0201-00045]
[Type: Rule]

Rule: Identifiers in preprocessor directives **must** be defined before use.

DAL:

A	B	C	D
req	req	req	adv

Reason: If an attempt is made to use an identifier in a preprocessor directive, and that identifier has not been defined, the preprocessor will sometimes not give any warning but will assume a value (usually zero).

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 19.11

Example: wrong:

```
#if FOO_C < 0  /* FOO assumed to be zero if not defined */
```

OK:

```
#define FOO_C 1
#if FOO_C < 0   /* FOO assumed to be zero if not defined */
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3.33 Preprocessor Directives Always Valid

ID: [LAT9-0201-00143]
[Type: Rule]

Rule: Preprocessor directives **must** be syntactically valid even when excluded by the preprocessor.

DAL:

A	B	C	D
req	req	req	adv

Reason: When a section of code is excluded by preprocessor directives, the content is ignored until a #else, #elif or #endif. If an excluded directive is badly formed, it may be ignored without warning by the compiler.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 19.16

Example: wrong:

```
#ifndef SYMBOL_C
    nVar1 = 0;
#else1                /* may be ignored by compilers! */
    nVar1 = SYMBOL_C;
#endif
```

OK:

```
#ifndef SYMBOL_C
    nVar1 = 0;
#else
    nVar1 = SYMBOL_C;
#endif
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.3.34 Preprocessor Directives “#else”, “#elif”, “#endif”

ID: [LAT9-0201-00144]
[Type: Rule]

Rule: All *#else, #elif* and *#endif* preprocessor directives **must** be located in the same file as the related *#if* or *#ifdef* directives.

DAL:

A	B	C	D
req	req	req	adv

Reason: If there are multiple inclusion and exclusion blocks, it can lead to confusion if not all directives occur within one single file. Following this rule preserves good code structure and avoids maintenance problems.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 19.17

Example: n/a

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.4 Function Usage Restrictions

7.3.4.1 Declaration of Function Parameters

ID: [LAT9-0201-00024]
[Type: Rule]

Rule: For each function parameter the type and the identifier in the declaration and definition **must** be always specified and identical. This applies also for empty parameters which **must be** specified with the type 'void' (that is return values and empty list of function argument).

This rule applies of course for the type of the return value which is a function (output) parameter as well.

DAL:

A	B	C	D
req	req	req	req

Reason: The declaration and the definition of each function must match for each argument and the return value, otherwise the behavior of compilers could be undetermined. This requires identical names and qualifiers (for function arguments) and types, and not just identical base types.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 8.3, 16.3, Rule 16.4, Rule 16.5

Example: wrong:

```
/* Declaration */
MyFunc1();

/* Definition */
void MyFunc1()
{
    :
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

```
}
```

OK:

```
/* Declaration */
void MyFunc1(void);

/* Definition */
void MyFunc1(void)
{
    :
}
```

wrong:

```
/* Declaration */
MyFunc2(uint32_t lVar1_i, sint32_t);

/* Definition */
uint16_t MyFunc(uint32_t lPar1_i, uint32_t lPar2_i)
{
    :
}
```

OK:

```
/* Declaration */
uint16_t MyFunc2(uint32_t lPar1_i, sint32_t lPar2_i);

/* Definition */
uint16_t MyFunc2(uint32_t lPar1_i, sint32_t lPar2_i)
{
    :
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.4.2 Needs of "return" Statement in Functions

ID: [LAT9-0201-00027]
[Type: Rule]

Rule: Each function **must** return a variable or a value, with the exception of a "void" function. Every exit path with a non-void return type **must** have an explicit return statement.

DAL:

A	B	C	D
req	req	req	req

Reason: Testability of the program increases.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 16.8

Example: wrong:

```
sint32_t MyFunc1(sint32_t lVar1_i, sint32_t lVar2_i)
{
    :
    return;
}
void MyFunc2(sint32_t lVar1_i, sint32_t lVar2_i)
{
    sint32_t lResult = 1;
    :
    return lResult;
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

OK:

```
sint32_t MyFunc1(sint32_t lVar1_i, sint32_t lVar2_i)
{
    sint32_t lResult = 1;
    :
    return lResult;
}
void MyFunc2(sint32_t lVar1_i, sint32_t lVar2_i)
{
    :
    return;
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.4.3 Number of "return" Statements in a Function

ID: [LAT9-0201-00133]
[Type: Rule]

Rule: There **must be** only one "return" statement, placed at the end of a function, if the function has a non-void return type. If the function has a void return type there **must not be** a “return” statement.

If macros ASSERT or ASSERT_ERR are defined and used like specified in [LAT9-0201-00130], their internal additional ‘return’ is not taken into account for this rule.

DAL:

A	B	C	D
req	req	req	req

Reason: A function is easier to understand if it contains just one exit point. The absence of a return statement leads to undefined behavior. When the return is placed at the end of the function, the code is easier to understand.

The principle of the ASSERT-macros reduces the complexity and improves the readability of the functions, increasing the maintainability and the quality of source code as well as the efficiency of development.
The need to have only one return is justified for the understanding of the algorithm. The use of more than one return can be allowed if this return is used for robustness improvement, limited to acting on function parameters. This is the goal of the ASSERT-macros.

Like required in the rule [LAT9-0201-00130], the invocation of the macros ASSERT and ASSERT_ERR must be placed as first statements in functions before any local variable definition or other statements. In this way, the understanding of the algorithm is not disturbed but improved by reducing its complexity.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

MISRA: Rule 14.7

Example: wrong:

```
bool_t MyFunc1(sint32_t lPar1_i, sint32_t * lRes_iop)
{
    sint32_t lVar;
:
    lVar = * lRes_iop;
    if (lPar1_i > 10000)   /* Parameter Validity */
    {
        return FALSE;
    }
:
    if (lPar1_i < lVar)
    {
        * lRes_iop = lVar;
        return TRUE;
    }
:
    return TRUE;
}
```

OK:

```
bool_t MyFunc1(sint32_t lPar1_i, sint32_t * lRes_iop)
{
    /* Parameter Validity */
    ASSERT_C(lPar_i <= 10000);
    ASSERT_C(lRes_iop NEQ_C NULL);
    {
        sint32_t lVar;
        bool_t bSuccess = FALSE;
:
        lVar = * lRes_iop;
:
        if (lPar1_i < lVar)
        {
            * lRes_iop = lVar;
            bSuccess = TRUE;
        }
:
        return bSuccess;
    }
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.4.4 Return Value of “void” Functions

ID: [LAT9-0201-00134]
[Type: Rule]

Rule: The values returned by *void* functions **must not** be used.

DAL:

A	B	C	D
req	req	req	req

Reason: This result in a void expression, the behavior is undefined.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: n/a

Example: wrong:

```
void MyFunc(sint32_t lVar1_i, sint32_t lVar2_i)
{
    :
}
lVar3 = MyFunc(lPara1, lPara2);
```

OK:

```
void MyFunc(sint32_t lVar1_i, sint32_t lVar2_i)
{
    :
}

MyFunc(lPara1, lPara2);
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.4.5 Function Call Parameters

ID: [LAT9-0201-00029]
[Type: Rule]

Rule: A function call **must** use all parameters that have been declared in the function prototype.

DAL:

A	B	C	D
req	req	req	adv

Reason: A function call using all the parameters means that they have all been defined and can be checked. The code is more readable and testable.
Consequently, the error rate decreases.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 16.6

Example: wrong:

```
/* Function prototype */
void Function1(sint32_t lAexample_i, sint32_t lBexample_i,
sint32_t lCexample_i);
:
/* Function call */
Function1(lAexample, lBexample);
```

OK:

```
/* Function prototype */
void Function_1(sint32_t lAexample_i, sint32_t
lBexample_i,
sint32_t lCexample_i);
:
/* Function call */
Function_1(lAexample, lBexample, lCexample);
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.4.6 Function Call without Defined Parameters

ID: [LAT9-0201-00139]
[Type: Rule]

Rule: Functions called with no parameters **must** have empty parentheses.

DAL:

A	B	C	D
req	req	req	adv

Reason: If the name of a function is used without parentheses, it represents a pointer to the function, not a call to the function.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 16.9

Example: wrong:

```
sint32_t MyFunc(void);

sint32_t main(void)
{
    MyFunc;
    ...
}
```

OK:

```
sint32_t MyFunc(void);

sint32_t main(void)
{
    MyFunc();
    ...
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.5 Include File

7.3.5.1 Definition and Declaration of Functions or Objects in Include Files

ID: [LAT9-0201-00074]
[Type: Rule]

Rule: Function and global object definitions **must not** be placed in an include file, but only their declaration if these elements are needed by other files. In this case, these declarations **must be** placed in one and only one header file.

For inline functions, which are together declaration and definitions, this rule does not apply if and only if the used compiler is C-99 compliant (see also [LAT9-0201-00014]).

DAL:

A	B	C	D
req	req	req	req

Reason: Header files are used to declare object, functions, typedefs and aliases, but not to define elements that occupy storage by themselves. Any element that occupies storage must be uniquely linked by the toolchain and therefore must not be placed in a file which may be included by other files. This separation makes it clear that only '.c' files contain executable source code and that '.h' files only contain declarations.

Any inline function in a '.h' file does not occupy storage in this '.h' file as compilation unit, but only in 'c' files where it is included (inserted) in the same way as macros. This is compliant to the MISRA Rule 8.5 and 8.8.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 8.5, Rule 8.8

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

Example: wrong:

```
/*  MyModule.h */

uint32_t lAGlobalVar_g = 1234;

void MyFunc(sint32_t lPar_i)
{
    ...
}
```

OK:

```
/*  MyModule.h – For declarations of public elements only */

extern uint32_t lAGlobalVar_g; /* Possible, if public need */

void MyFunc(sint32_t lPar_i); /* Only function prototype here */

/* Allowed: inline function (if compiler C99 compliant) */
inline uint32_t CalculateFast(sint32_t lParam_i)
{
    ...
}


/*  MyModule.c */

#include "MyModule.h"  /* Declarations of public elements */

uint32_t lAGlobalVar_g = 1234;  /* Definition of the object */

void MyFunc(sint32_t lPar_i)
{
    ...
    lRes = CalculateFast(lPar_i);
    ...
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.5.2 “Typedefs” in Include Files

ID: [LAT9-0201-00075]
[Type: Rule]

Rule: Each typedef **must be** placed in one include file only if this type is intended to be used in more than just one file.

DAL:

A	B	C	D
req	req	req	adv

Reason: Makes code easier to maintain.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: n/a

Example:

wrong:

file.h:

```
:
typedef struct my_private_type_tag /* only used in file.c */
{
    sint32_t lValue1;
    sint32_t lValue2;
} my_private_type_t;
```

file.c:

```
#include "file.h"
:
my_private_type_t MyPrivateObject;
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

OK:

file.c:

```
typedef struct my_private_type_tag
{
    sint32_t lValue1;
    sint32_t lValue2;
} my_private_type_t;
:
my_private_type_t MyPrivateObject;
:
```

OK:

file.h:

```
:
/* used in file1.c and file1.c */
typedef struct my_private_type_tag
{
    sint32_t lValue1;
    sint32_t lValue2;
} my_private_type_t;
```

file1.c:

```
#include "file.h"
:
my_private_type_t MyPrivateObject;
:
```

file2.c:

```
#include "file.h"
:
my_private_type_t MyPrivateObject;
:
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.5.3 Recursive/Repetitive Inclusion of Include Files Content

ID: [LAT9-0201-00078]
[Type: Rule]

Rule: The multiple loading of an *include* file content during a compilation must not occur.

DAL:

A	B	C	D
req	req	req	adv

Reason: It is compiler dependent. There is no need to load include files multiple times because it can be avoided with a *#ifndef __[NAME]_H__*, which checks if the include file is already loaded. As a consequence, compiler performance improves.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 19.15

Example: wrong:

```
/* global.h */
#define MY_DEFINE_C 10
/* EOF */

/* module_a.h */
:
#include "global.h"
:

/* module_a.c */
:
#include "module_a.h"
#include "global.h"     /* is already included */
:
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

OK:

```
/* global.h */
#ifndef __GLOBAL_H__  /* test for multiple inclusion */
#define __GLOBAL_H__
#define MY_DEFINE_C 10
#endif                /* end of test for multiple inclusion */
/* EOF */
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.6 Operators

7.3.6.1 Forbidden Operators

ID: [LAT9-0201-00080]
[Type: Rule]

Rule: The following operators:

```
==
!=
||
&&
!
```

must not be used - they **must** be defined with a logical name in a *define* preprocessor command.

DAL:

A	B	C	D
req	req	req	req

Reason: Improved readability of the code and less confusion with other operators such as =, |, &.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: n/a

Example: wrong:

```
if (nTest == 0)
:
```

OK:

```
#define EQU_C ==
#define NEQ_C !=
#define OR_C ||
#define AND_C &&
#define NOT_C !

if (nTest EQU_C 0)
:
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.6.2 Use of "? :" Operator

ID: [LAT9-0201-00081]
[Type: Rule]

Rule: The *? :* operator **must not** be used.

DAL:

A	B	C	D
req	req	req	adv

Reason: The "?:" operator is difficult to test, can reduce the readability dramatically, and offers no advantages in comparison with of using an *if-else* statement.

Applicability:

SDDM1	SDDM2	SDDM3
yes	No	yes

MISRA: n/a

Example: wrong:

```
nMax_ab = (nVal_a > nVal_b) ? nVal_a : nVal_b;
```

OK:

```
if (nVal_a > nVal_b)
{
    nMax_ab = nVal_a;
}
else
{
    nMax_ab = nVal_b;
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.6.3 Use of the “unary minus” Operator

ID: [LAT9-0201-00082]
[Type: Rule]

Rule: The unary minus operator shall not be applied to an unsigned expression.

DAL:

A	B	C	D
req	req	req	adv

Reason: Applying the unary minus operator to an unsigned expression is not a particularly meaningful operation to perform, except in a mixed signed/unsigned expression (which itself should be avoided). In certain circumstances and on some compilers it can produce indeterminate behaviour.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 12.9

Example: wrong:

```
uint32_t lVar_Unsigned;
uint16_t nVar_Unsigned = 1;

lVar_Unsigned = -nVar_Unsigned;
```

OK:

```
sint32_t lVar_Signed;
sint16_t nVar_Signed = 1;

lVar_Signed = -nVar_Signed;
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.6.4 Use of the “comma” Operator

ID: [LAT9-0201-00083]
[Type: Rule]

Rule: The *comma* operator **must not** be used. The comma separator (declaration of enumerations, initialization of arrays or data structure, etc.) is allowed.

DAL:

A	B	C	D
req	req	adv	adv

Reason: Use of the *comma* operator is generally detrimental to the readability of code, and the same effect can be achieved by other means.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 12.10

Example:

wrong:

```
uint32_t lVar1, lVar2, lArray_a[10], *lPtr_p;
...
while (nVal1++, (nVal2 > 0))
...
for (i = 0, nVal1 = TEN_C; i < nVal1; i++, nVal1--)
...
```

OK:

```
typedef enum MyEnum_tag
{
    Enum1_E,
    Enum2_E,
    :
} MyEnum_t;
...
uint8_t cMyArray_a[3] = {1, 2, 3};
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.6.5 Use of the “shift” Operator

ID: [LAT9-0201-00084]
[Type: Rule]

Rule: The right hand operand of a *shift* operator **must** be between 1 and one less than the width in bits of the left hand operand (inclusive).

DAL:

A	B	C	D
req	req	req	adv

Reason: Shifting a variable by more bits than its length might lead to undefined behaviour.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 12.8

Example:

```
uint8_t cVal1;
uint8_t cVal2;
uint16_t nVar1 = 0xFFFF;
uint32_t lVar2;
```

wrong:

```
cVal2 = cVal1 << TEN_SHIFT_C;
```

OK:

```
cVal2 = cVal1 << SEVEN_SHIFT_C;
...
lVar2 = ((uint32_t) nVar1) << SIXTEEN_SHIFT_C;
                              /* nVar2 = 0xFFFF 0000 */
...
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.6.6 Use of the “sizeof” Operator

ID: [LAT9-0201-00085]
[Type: Rule]

Rule: The *sizeof* operator **must not** be used on expressions that contain side effects.

DAL:

A	B	C	D
req	req	req	req

Reason: A common programming error in C is to apply the *sizeof* operator to an expression and expect the expression to be evaluated. However the expression is not evaluated: sizeof only acts on the type of the expression. To avoid this error, *sizeof* must not be used on expressions which contain side effects, as the side effects will not occur.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 12.3

Example: wrong:

```
... sizeof(++nVal1) ...;

/* "nVal1 = 0" will not be executed - sizeof only returns
the size of "nVal1". */
```

OK:

```
++nVal1;
... sizeof(nVal1) ...;
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.6.7 Bitwise Operations Must Be cast

ID: [LAT9-0201-00086]
[Type: Rule]

Rule: The result of a bitwise operation **must be** cast immediately after calculation and before using it.

DAL:

A	B	C	D
req	req	req	adv

Reason: When the operators '~' and '<<' are applied to *small integer types* (= strictly smaller to the standard one of the platform), the operations are preceded by integral promotion, and the result may contain high order bits which have not been anticipated.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 10.5

Example: wrong:

```
uint8_t cRes;
cRes = (~0x5A) >> 4;    /* !!! Can give 0xFA, not 0x0A ! */
```

OK:

```
uint8_t cRes;
cRes = (uint8_t)((uint8_t)(~0x5A)) >> 4;  /* OK, 0x0A */
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.6.8 Bitwise Operations Only On Unsigned Integer

ID: [LAT9-0201-00151]
[Type: Rule]

Rule: Bitwise operations **must not** be performed with signed integer types if the CPU does not implement bitwise operations as arithmetic operations.

When using the bitwise operations with signed integer types, the developer must demonstrate that its platform supports arithmetic operations.

DAL:

A	B	C	D
req	req	req	adv

Reason: Bitwise operations are not meaningful on signed integers and behavior is not clearly defined, therefore the results of these operations are platform-dependent.
Problem can arise if e.g. a right shift moves the sign bit into the number, or a left shift moves a numeric bit into the sign bit.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 12.7

Example: wrong:

```
sint32_t lVal1;
:
lVal1 &= 0x7F7F7F7F;   /* !!! signed => undefined behavior */
```

OK:

```
uint32_t lVal1;
:
lVal1 &= 0x7F7F7F7F;   /* OK, unsigned value */
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.6.9 Logical Operators and Side Effects

ID: [LAT9-0201-00137]
[Type: Rule]

Rule: The right hand operand of the && or || operator **must not** contain side effects.

DAL:

A	B	C	D
req	req	req	adv

Reason: There are some situations in C code where certain parts of expressions may not be evaluated.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 12.4

Example: wrong:

```
if ((lVal1 EQU_C lVal2) AND_C ((lVal3 = Func(lVal4)) EQU_C
lVal5))
```

OK:

```
lVal3 = Func(lVal4);
if ((lVal1 EQU_C lVal2) AND_C (lVal3 EQU_C lVal5))
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.6.10 Logical Operators Need Primary Expressions

ID: [LAT9-0201-00148]
[Type: Rule]

Rule: The operands of a logical && or || operator **must** be primary expressions.

DAL:

A	B	C	D
req	req	req	adv

Reason: The effect of this rule is to require that if an operand is other than a single identifier or constant then it must be parenthesized. This will increase readability and will ensure that the behavior is as the programmer intended.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 12.5

Example: wrong:

```
bResult = bVal1 AND_C bVal2 OR_C bVal3;
bResult = bVal1 OR_C bVal2 AND_C bVal3;
bResult = NOT_C bVal1 AND_C bVal2;
```

OK:

```
bResult = (bVal1 AND_C bVal2) OR_C bVal3;
bResult = bVal1 OR_C (bVal2 AND_C bVal3);
bResult = NOT_C (bVal1 AND_C bVal2);
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.6.11 Assignment Operators

ID: [LAT9-0201-00087]
[Type: Rule]

Rule: Assignment operators **must not** be used in expressions which produce Boolean values.

DAL:

A	B	C	D
req	req	req	adv

Reason: If assignments are required in the operands of a Boolean-valued expression then they must be performed separately outside of those operands. This helps to avoid getting “=” and “==” confused, and assists the static detection of errors.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	yes

MISRA: Rule 13.1

Example: wrong:

```
if ((nVal1 = nVal2) NEQ_C 0)
{
    ...
}
```

OK:

```
nVal1 = nVal2;
if (nVal1 NEQ_C 0)
{
    ...
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

7.3.6.12 Operator Precedence

ID: [LAT9-0201-00089]
[Type: Rule]

Rule: No dependence **must** be placed on C's operator precedence rules in expressions for cases other than the ones only with mathematical operators (+, -, *, /). Use parenthesis to set precedence of expressions.

DAL:

A	B	C	D
adv	adv	adv	adv

Reason: It is easy to make a mistake with the rather complicated precedence rules of C, and this approach helps to avoid such errors, and helps to make the code easier to read.

Applicability:

SDDM1	SDDM2	SDDM3
yes	no	no

MISRA: Rule 12.1

Example:

wrong:

```
nVar1 = nVar2 & MASK_C >> 2 * nVar3;
...
bVar1 = NOT_C bVar2 AND_C bVar3 OR_C bVar4;
```

OK:

```
nVar1 = nVar2 & (MASK_C >> (2 * nVar3));
...
bVar1 = ((NOT_C bVar2) AND_C (bVar3 OR_C bVar4));
...
nVar1 = nVar2 * nVar3 + nVar2 / nVar4;
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

8 Guidelines

This chapter contains several common practices which are not mandatory but will guide to a better handling of software, improving efficiency of development, of verification activities and maintenance actions. Every programmer should apply them.

8.1 Layout of Source File Headers

ID: [LAT9-0201-00005]
[Type: Guideline]

Guideline: Programmers **should** use a header for each file and each function / operation. The used layout can vary but must be the same within a single project and must fulfill the following aspects:

Readability and Transparency: The different parts of the code (e.g. includes, defines, function code) must be clearly recognizable.
Usability: The header layout must only contain passages that are related to the specific file type. This means e.g. for normal C-code, that for .c and .h files different variations of the same header must be used.
Identification: Code specific information like title, project and module name must appear at the beginning of every source file.
Description: For every function, operation, typedef and global variable/object, an adequate description deducted from the design must exist to help the developers to understand these elements for programming or maintenance actions.
Copyright: Each module header must contain a copyright declaration, especially for modules not created by a code generator.

An example of header layout can be found in the example below.

Reason: An uniform layout for all modules improves readability and comprehensibility of the module and thereby the global efficiency of the project will increase, the development workload will be reduced.

MISRA: n/a

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

Example: OK:

Header for .c-file:

```
/*****************************************************************************/
/* MODULE HEADER                                                             */
/*****************************************************************************/
/*
  MODULE:      Aileron
  PROJECT:     RRJ ACE_II Control Channel
  SVN Info:    $Id$

  DESCRIPTION:
  This module contains the surface-specific functions for Aileron.

  Copyright (C) by Liebherr-Aerospace Lindenberg GmbH
*****************************************************************************/

/*****************************************************************************/
/* INCLUDES                                                                  */
/*****************************************************************************/
#include "Identity.h"
#include "ErrSys.h"

/*****************************************************************************/
/* FUNCTION PROTOTYPE                                                        */
/*****************************************************************************/

/* ---- public operations ------------------------------------------------- */
#include "Aileron.h"

/* ---- private operations ------------------------------------------------ */
bool_t Aileron_PerformPrimaryTask (void);

bool_t Aileron_CalcDirectControlLaws (Aileron_t * const This_iocp,
                                      float32_t fActuatorCmdCons_i);

/*****************************************************************************/
/* CLASSES / STRUCTURES / TYPES / DEFINEs (private)                          */
/*****************************************************************************/
/* ---- DEFINEs from <<embedded>> classes --------------------------------- */
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

```
/* ---- private embedded classes ---------------------------------------- */

/*****************************************************************************/
/* SPECIAL MODULE DEFINITIONS                                                */
/*****************************************************************************/
#define AILERON_MAGIC_NUMBER_C ((uint32_t)0x228D1DDEU)

/*****************************************************************************/
/* OBJECTS / INSTANCES OF CLASSES (supplied by the class)                    */
/*****************************************************************************/

/*___ OBJECT: Aileron_____________________________________________________*/
Aileron_t Aileron_g;

/*****************************************************************************/
/* OPERATION DESCRIPTION 'Aileron_Init'                                      */
/*****************************************************************************/
/*
  PURPOSE:
      Initialize the Aileron module

  ADDITIONAL INFORMATION:                          OPERATION KIND: Functional

  PARAMETERS:
      IN:
      OUT:
      IN/OUT:
          This : Aileron    {'This' must reference an existing object}
             Reference of the object used in the operation
  RETURN:
      bool      { - No validity description - }
          TRUE = success, FALSE = failure

  USED GLOBAL VARIABLES/OBJECTS:
      IN:
      OUT:
      IN/OUT:

  REALTIME CONSIDERATIONS:                         REENTRANT: No
*****************************************************************************/
bool_t Aileron_Init (Aileron_t * const This_iocp)
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

```
{
....

Header for .h-file:
/*****************************************************************************/
/* INCLUDE HEADER                                                            */
/*****************************************************************************/
/*
  MODULE:      Aileron
  PROJECT:     RRJ ACE_II Control Channel
  SVN Info:    $Id$

  DESCRIPTION:
  This module contains the surface-specific functions for Aileron.

  Copyright (C) by Liebherr-Aerospace Lindenberg GmbH
*****************************************************************************/

#ifndef __AILERON_H__  /* test for multiple inclusion */
#define __AILERON_H__

/*****************************************************************************/
/* INCLUDES                                                                  */
/*****************************************************************************/

/* ---- From supplier classes / modules ---------------------------------- */
#include "StdDefs.h"
#include "ChannelSync.h"
#include "PfcuInputVoter.h"

/* ---- From sub-classes ------------------------------------------------- */

/*****************************************************************************/
/* CLASSES / STRUCTURES / TYPES / DEFINES                                    */
/*****************************************************************************/

/* ---- DEFINEs from <<embedded>> classes -------------------------------- */

/* ---- Basic Types ------------------------------------------------------ */

/* ---- Embedded Classes ------------------------------------------------- */
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

```
typedef struct AileronSSFaults_tag
{
  ...
} AileronSSFaults_t;
...
/* ---- Master Class ------------------------------------------------------ */
typedef struct Aileron_tag
{
    float32_t                  fActuatorDMCmd;
    ...
} Aileron_t;

/*****************************************************************************/
/* OBJECTS / INSTANCES OF CLASSES (supplied by the class)                    */
/*****************************************************************************/
extern Aileron_t Aileron_g;

/*****************************************************************************/
/* OPERATION PROTOTYPES                                                      */
/*****************************************************************************/
bool_t Aileron_Init (Aileron_t * const This_iocp);

bool_t Aileron_Reset (Aileron_t * const This_iocp);

bool_t Aileron_ReadInputs (Aileron_t * const This_iocp);

bool_t Aileron_WriteOutputs (Aileron_t * const This_iocp);

bool_t Aileron_Processing (Aileron_t * const This_iocp);

#endif /*__AILERON_H__*/

/* EOF */
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

8.2 Indent Every New Block with 4 Blanks (Spaces)

ID: [LAT9-0201-00008]
[Type: Guideline]

Guideline: If a new block starts, then the code **should be** indented with 4 blanks.

Reason: Readability of the code increases and helps a reviewer understand the code better.

MISRA: n/a

Example: wrong:

```
if ( nVar1 < MAXVAL_C )
{
nVar2 += nVar1;
nResult = MAXVAL_C / nVar2;
}
if ( nResult >= 1 )
{
bAction = TRUE;
}
```

OK:

```
if( nVar1 < MAXVAL_C )
{
    nVar2 += nVar1;
    nResult = MAXVAL_C / nVar2;
}
if( nResult >= 1 )
{
    bAction = TRUE;
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

8.3 Indentation with Tabulator

ID: [LAT9-0201-00103]
[Type: Guideline]

Guideline: All indentation done with tabulator **should be** converted to 4 blanks.

Reason: Indentation has no standard width and will be interpreted differently depending of the used editor.

MISRA: n/a

Example: wrong:

```
if (nVar2 > nVar1)
{
    nVar2 = 0; /* Indentation with 1 tab */
}
```

OK:

```
if (nVar2 > nVar1)
{
    nVar2 = 0; /* Indentation with 4 blanks */
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

8.4 Horizontal Spacing

ID: [LAT9-0201-00105]
[Type: Guideline]

Guideline:

(a) A blank (space) **should** be inserted before and after the following delimiters : +, -, / , *, &, |, <, <=, ..., >, >=, = (also for EQU_C, etc.)

(b) Do not insert blanks before the semi-colon ";".

(c) Do not leave spaces before or after "." and "->".

(d) Expressions and argument lists **should** be separated by one blank.

(e) Do not leave spaces after the address-operator "&".

Reason: Makes code easier to understand because of the standard layout.

MISRA: n/a

Example:

a)
wrong:

```
if((aEQU_Cb)AND_C(cEQU_Cb))
*nTemp_op=nTemp_i;
```

OK:

```
if((a EQU_C b) AND_C (c EQU_C b))
*nTemp_op = nTemp_i;
```

b)
wrong:

```
*nTemp_op = nTemp_i ;
```

OK:

```
*nTemp_op = nTemp_i;
```

c)
wrong:

```
My_Struct . nValue = TEN_C;
My_Struct_p -> nValue = TEN_C;
```

OK:

```
My_Struct.nValue = TEN_C;
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

```
My_Struct_p->nValue = TEN_C;
```

d)
wrong:

```
for(i = 0;i < 1;i++)
Function(i,j);
```

OK:

```
for (i = 0; i < 1; i++)
Function(i, j);
```

e)
wrong:

```
nPointer_p = & nVar1;
```

OK:

```
nPointer_p = &nVar1;
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

8.5 Pointer to Constant as Function Parameter

ID: [LAT9-0201-00102]
[Type: Guideline]

Guideline: Const qualification **should be** used on pointers as function parameters which are passed by reference, where it is intended that the function will not modify the parameter value.

Reason: This guideline helps to prevent the modification of a function parameter which is intended as input only, and also serves to indicate to the reader that it is the intention.

MISRA: Rule 16.7

Example: wrong:

```
/* Only input param., but value can be modified */
sint32_t MyFunc(sint32_t * const lPointer_ip)
{
    :
    sint32_t lResult = *lPointer_ip
    return lResult;
}
```

OK:

```
/* Input parameter: value can not be modified */
sint32_t MyFunc(sint32_t const * const lPointer_icp)
{
    :
    sint32_t lResult = *lPointer_icp
    return lResult;
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

8.6 Constant Pointer as Function Parameter

ID: [LAT9-0201-00158]
[Type: Guideline]

Guideline: Const qualification **should be** used on pointers as function parameters to prevent the address pointed to is modified.

Reason: This guideline helps to prevent the modification of the address the pointer points to as the developer in the calling function might not be aware that the pointer directs to another address after calling the function.

MISRA: -

Example: wrong:

```
/* Only input param., but address can be modified */
sint32_t MyFunc(sint32_t const *lPointer_ip)
{
    :
    sint32_t lResult = *lPointer_ip
    return lResult;
}
```

OK:

```
/* Input parameter: address can not be modified */
sint32_t MyFunc(sint32_t const * const lPointer_icp)
{
    :
    sint32_t lResult = *lPointer_icp
    return lResult;
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

8.7 Expression Bracket Depth Exceeding 7

ID: [LAT9-0201-00033]
[Type: Guideline]

Guideline: The maximum bracket depth in an expression **should not** exceed 7 (seven).

Remark : Expressions with a high arithmetic complexity should be avoided wherever possible.

Reason: A large bracket depth is very difficult to read and is a high source of programming errors. Complexity is increased and reliability reduced.

MISRA: n/a

Example: wrong:

```
nResult = (nFaktor * (nSum +((nDn + EIGHT_C * (((((((
           nAn + nBn)))))))))));
```

OK:

```
nResult = (nFaktor * (nSum + (nDn + EIGHT_C) * (nAn + nBn)));
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

8.8 Prefixes and Postfixes of Identifiers

ID: [LAT9-0201-00095]
[Type: Guideline]

Guideline: The variable identifier of simple data types **should have** the form like shown below (for short life variables like loop counters, it may be released):

```
< data type prefix > < name > [ < postfix > ]
```

For better readability of the code it is efficient to identify the data type of a variable by the name. So each variable has a short data type prefix. The foundation of the data type prefixes is known as "Hungarian Notation".

Type	Prefix	Example
s/uint8_t	c	cInputKey
s/uint16_t	n	nVar1
s/uint32_t	l	lCounter
s/uint64_t	ll	llValue
float32_t	f	fResult
float64_t	d	dTemp
string_t	s	sName
bool_t	b	bError

The postfix (also known as suffix) of an identifier (variable, function argument) gives additional information. It is separated by the underscore character "_" and can be a combination of several suffixes. In this case the order of the suffixes from the left to the right is equal to the top-down order in this table (e.g. nSpeed_cag). A field of characters is normally used as a string, meaning with the prefix "s", not with the suffix "_a".

Type	Letters	Postfix	Example
function input parameter	upper / lowercase	i	cSign_i
function output parameter	upper / lowercase	o	nNumbers_o
function input/output paramter	upper / lowercase	io	nValue_io
constant	upper / lowercase	c	nMy_Const_c
pointer	upper / lowercase	p	nValue_p
function pointer	upper / lowercase	pfn	My_Function_pfn
array	upper / lowercase	a	nValues_a
global variable	upper / lowercase	g	nCount_g
local variable	upper / lowercase	-	nCount

Reason: This will increase readability and testability.

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

MISRA: n/a

Example: OK:

```
sint32_t         *lValue_pg;          /* global pointer to long int */
sint32_t          lValues_a[100];     /* long integer array */
bool_t           *bError_pg;          /* global pointer to boolean */
uint8_t           sName_g[100];       /* global string */
sfloat32_t       *fValue1_p;          /* pointer to float */
const sint32_t    lMy_Const_cg;       /* global constant integer */
sint32_t         *lValues_pag[100];   /* global pointer to an
                                         long integer array */
sint32_t * const  lValues_cpg;        /* global constant
                                         pointer to a long integer */
```

OK:

```
sint32_t         *lValue_pg;        /* global pointer to long int */
sint32_t          lValues_a[100];   /* long integer array */
bool_t           *bError_pg;        /* global pointer to boolean */
uint8_t           sName_g[100];     /* global string */
sfloat32_t       *fValue_1_p;       /* pointer to float */
const sint32_t    lMy_Const_cg;     /* global constant integer */
sint32_t         *lValues_pag[100]; /* global pointer to an
                                       long integer array */
sint32_t * const  lValues_cpg;      /* global constant
                                       pointer to an long integer */
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

8.9 Location of Function Prototypes

ID: [LAT9-0201-00106]
[Type: Guideline]

Guideline: Each declaration of function (prototype) **should be** placed after any definition of types or defines and before any definition of functions or objects.

Reason: The compiler does not check the parameters if no function prototyping is used. The placement is important to make code easier to maintain and avoids inconsistencies.

MISRA: n/a

Example: wrong:

```
module.h:
:
void MyPrivateFunction(void); /* only used in module.c */

module.c:
#include "module.h"

void MyPrivateFunction(void)
{
:
}
```

OK:

```
module.c:
void MyPrivateFunction(void); /* only used in module.c */
:
void MyPrivateFunction(void)
{
:
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

wrong:

module.c:

```
uint32_t AnObject;
:
void AFunction(void)
{
:
}
:
void MyPrivateFunction(...); /* Wrong place for a prototype */
:
typedef struct my_private_type_tag
{
    sint32_t lValue1;
    sint32_t lValue2;
} my_private_type_t;
:
```

OK:

module.c:

```
typedef struct my_private_type_tag
{
    sint32_t lValue1;
    sint32_t lValue2;
} my_private_type_t;
:
void MyPrivateFunction (...); /* OK */
:
uint32_t AnObject;
:
void AFunction(...)
{
:
}
:
void MyPrivateFunction(void)
{
:
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

8.10 Operation Naming Conventions for OOP

ID: [LAT9-0201-00107]
[Type: Guideline]

Guideline: For OOP the operation names **should** begin with the class name (type without suffix), followed by a "_" and the real operation name.

Reason: By using this name format, the parent class of the operation can be easily recognized by the operation name.

MISRA: n/a

Example: wrong:

```
class:      DrvFlash
operation:  Init(...)
```

OK:

```
class:      DrvFlash
operation:  DrvFlash_Init(...)
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

8.11 Maximum of 150 Characters per Line

ID: [LAT9-0201-00108]
[Type: Guideline]

Guideline: Code **should not** have more than 150 characters per line.
Comments following at the end of a code line are not counted.

Reason: The program is more readable and understandable.

MISRA: n/a

Example: n/a

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

8.12 Storage Type "register"

ID: [LAT9-0201-00091]
[Type: Guideline]

Guideline: The use of the keyword “register” **should be** avoided.

Reason: Each compiler has internal rules for the usage of the internal processor registers. Do use ‘register’ unless you have good reasons for telling the compiler to locate a specific variable in an internal processor register. Moreover, ISO compilers are not obliged to take any notice of the "register" storage class specifier, so at best it is only a hint to the compiler. A good optimiser should be able to decide for itself what to put in registers.

MISRA: n/a

Example: -

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

8.13 Maximum Block Depth Limited to 6

ID: [LAT9-0201-00022]
[Type: Guideline]

Guideline: The maximum block depth in any iterative type of structure is 6 - this **should not** be exceeded.

Reason: Makes code easier to read, comprehend and test.

MISRA: n/a

Example: wrong:

```
if (nVar_a < 0)
{
    if (nVar_b < 0)
    {
        if (nVar_c < 0)
        {
            for (i = 0; i <= TEN_C; i++)
            {
                if (i * nVar_a >= TEN_C)
                {
                    while (nVaR_a * nVar_b < 0)
                    {
                        if (nVar_c EQU_C 0)
                        {
                            nResult++;
                        }
                    }
                }
            }
        }
    }
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

OK:

```
if (nVar_a < 0)
{
    if (nVar_b < 0)
    {
        for (i = 0; i <= TEN_C; i++)
        {
            nHelp++;
        }
    }
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

8.14 Valid Pointers

ID: [LAT9-0201-00109]
[Type: Guideline]

Guideline: Never assume you have a valid pointer. Pointers **should** check always for NULL before dereferencing a pointer, unless you can prove it cannot be NULL.

Reason: This will increase the quality of the code and its robustness.

MISRA: n/a

Example: n/a

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

8.15 No Source Code “Commented Out”

ID: [LAT9-0201-00125]
[Type: Guideline]

Guideline: Sections of code **should not** be "commented out".

Reason: Disturb the readiness of the program and a review of the code against the SW-design.

MISRA: Rule 2.4

Example: wrong:

```
bStatus_o = TRUE;
/* bStatus_o = FALSE; */

OK:
bStatus_o = TRUE;
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

8.16 Overflow of Unsigned Integer with Constant

ID: [LAT9-0201-00050]
[Type: Guideline]

Guideline: Evaluation of constant unsigned integer expressions **should not** lead to wrap-around.

Reason: Because unsigned integer expressions do not strictly overflow, but instead wrap around in a modular way, any constant unsigned integer expression which effect "overflow" will not be detected by the compiler.

MISRA: Rule 12.11

Example: On a machine with 16-bit *int* type and a 32-bit *long* type:

wrong:

```
#define START 0x8000U
#define END 0xFFFF0000U
#define LEN 0x8000U

#if ((START + LEN) > END)
  #error Buffer Overrun
  /* Not OK: START + LEN wraps around to 0x0000 */
#endif
```

OK:

```
#define START 0x8000UL
#define END 0xFFFF0000UL
#define LEN 0x8000UL

#if ((START + LEN) > END)
  #error Buffer Overrun
  /* OK because START and LEN are unsigned long
#endif
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

8.17 Complex Type Definition with Tag

ID: [LAT9-0201-00136]
[Type: Guideline]

Guideline: Every new defined complex type (with typedef) **should have** an identifier pseudo name, which ends with "_tag".

Reason: This will increase readability and testability. In some times, the tag is needed (declaration of attribute whose type is the current declared one like for a chained list).

MISRA: n/a

Example: OK:

```
typedef struct MyType_tag
{
    sint32_t lVal1;
    sint32_t lVal2;
} MyType_t;
:
MyType_t MyVariable;
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

8.18 Preprocessor Directive “#endif” with Comments

ID: [LAT9-0201-00145]
[Type: Guideline]

Guideline: Every occurrence of the #endif directive **should** be commented, to clarify, which #if or #ifdef block is closed.

Reason: This increases readability when code is excluded, especially when multiple #if-directives are used.

MISRA: n/a

Example: wrong:

```
#ifdef DEBUG_C
...
#ifdef DIAGNOSE_C
...
#endif               /* which #endif to which #ifdef?? */
#endif
```

OK:

```
#ifdef DEBUG_C
...
#ifdef DIAGNOSE_C
...
#endif /* ifdef DIAGNOSE_C */
#endif /* ifdef DEBUG_C */
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

8.19 Definition of Basic Types

ID: [LAT9-0201-00150]
[Type: Guideline]

Guideline: The basic types of char, short, long and float **should be** replaced by the following type definition in order to not use directly the basic types and to use the correct types for bitfields like required in LAT9-0201-00047 and LAT9-0201-00149:

```
 8 Bit signed   integer:      sint8_t
 8 Bit unsigned integer:      uint8_t
16 Bit signed   integer:      sint16_t
16 Bit unsigned integer:      uint16_t
32 Bit signed   integer:      sint32_t
32 Bit unsigned integer:      uint32_t
64 Bit signed   integer:      sint64_t
64 Bit unsigned integer:      uint64_t
32 Bit floating point:        float32_t
64 Bit floating point:        float64_t
128 Bits floating point:      float128_t
Boolean type:                 bool_t
```

Type definitions for bitfields:

```
typedef signed int      sbitfield_t;  /* Signed bitfield */
typedef unsigned int    ubitfield_t;  /* Unsigned bitfield */
```

Type definition for character values:

```
typedef   char     char_t;
typedef   char*    string_t;
```

Note 1:
The types `unsigned char` and `signed char` **should not** be used for character values but is reserved to numeric values.

Note 2:
The definition of the boolean type `bool_t` is depending on the capabilities of the platform and the compiler extension for C99 boolean type:

`uint8_t` : for platform supporting 8 bits data as smallest size
`uint16_t` : for platform supporting 16 bits data as smallest size
`_Bool` : for compiler supporting C99 'like' `_Bool` extension

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

Reason: The storage length of types can vary from compiler to compiler. It is safer if programmers work with types which they know to be of a given length or properties.
The use of a boolean type makes the code more understandable on the intention of use of variables and statements.

MISRA: Rule 6.1, Rule 6.2, Rule 6.4

Example: wrong:

```
typedef struct adc_error_bit_tag
{
   signed int    StdError:4;
   :
} adc_error_bit_t;

signed char cVal1 = 1;
unsigned long lVal2 = 0xffffffff;
signed char cMyBooleanMeaningSuccess = 1;
```

OK:

std defs.h

```
typedef signed char     sint8_t;
typedef unsigned long   uint32_t;
typedef unsigned char   bool_t; /* Platform supports 8 bits */
#define TRUE  ((bool_t) 1)
#define FALSE ((bool_t) 0)
typedef struct adc_error_bit_tag
{
   sbitfield_t    StdError:4;
   :
} adc_error_bit_t;
```

Alternative for bool_t with C99-like boolean:

```
#define bool_t _Bool  /* Compiler supports C99-like boolean */
#define TRUE true     /* Compiler supports C99-like boolean */
#define FALSE false   /* Compiler supports C99-like boolean */
```

c-file

```
#include "std_defs.h"
:
sint8_t  cVal1 = 1;
uint32_t lVal2 = 0xffffffff;
bool_t bSuccess = TRUE;
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

8.20 Floating Point Integration

ID: [LAT9-0201-00068]
[Type: Guideline]

Guideline: Integration of floating point values **should not** be performed.

Reason: The integration of 32bit floating point values generates failures in the resulting value. Keep in mind, that a 32bit floating point value represents a real numeric range of 2^{-24} around the value (this is the failure due to the [IEEE-754] representation of a value). As an integration is done via a successive adding of a small value, this minimal imprecision (the '2^{-24} around the value') appears at each step. The sum of this imprecision during the iteration makes the impact on the result perceptible.
Against an integration, a successive direct calculation avoid the adding of the imprecision at each step, and this imprecision is reduced to one time the '2^{-24} around the value' at any step of the calculation.

MISRA: n/a

Example:

```
sfloat32_t fVar;
sint32_t i;
```

wrong:

```
fVar = 0.0;
for (i = 1; i <= TEN_THOUSAND_C; i++)
{
    fVar = fVar + 0.1;
    ...
}
```

OK:

```
for (i = 1; i <= TEN_THOUSAND_C; i++)
{
    fVar = i * 0.1;
    ...
}
```

Note:
The wrong case results in fVar = 999.902893066406250.
This is a absolute Failure of dx = 0.0971!
The good case calculates the correct result fVar = 1000.000000000000000.

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

8.21 Separate Lines for the Block Borders "{" And "}"

ID: [LAT9-0201-00010]
[Type: Guideline]

Guideline: The start-block sign "{" and the end-block sign "}" **should be** placed at the same column (same indentation) but in a new line where no other code is placed (comments are allowed).

- Array initialization may not follow this guideline
- At the do-while statement, this guideline does not apply because the ‘while' must be placed on the same line as the closing bracket '}’.

Reason: Readability of the source is increased because the block structure is clearer.

MISRA: n/a

Example: wrong:

```
if (nVar1 EQU_C nVar2) {
nVar3 = nVar1 * nVar2;
nVar1 = nVar3 + nVar2; }
```

OK:

```
if (nVar1 EQU_C nVar2)
{
    nVar3 = nVar1 * nVar2;
    nVar1 = nVar3 + nVar2;
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

8.22 Comparison Operators for End Condition In "for" Loop

ID: [LAT9-0201-00118]
[Type: Guideline]

Guideline: For an end condition the operators <, <=, >, >= **should** be used instead of ==, !=.

Reason: Loops must be implemented in such a way that every programmer/reviewer easily and clearly understands the nature and function of the loop, and that the loop could stops also in case of error due to memory leaks or misusing of the loop index.
Moreover, in case of floating-point comparison, the comparator “==” and “!=” may not perform the expected result due to the internal representation of the values.

MISRA: n/a

Example: wrong:

```
for (i = TEN_C; i NEQ_C 0; i--)
{
:
}
```

OK:

```
for (i = TEN_C; i > 1; i--)
{
:
}
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

8.23 Naming Convention of Identifiers

ID: [LAT9-0201-00097]
[Type: Guideline]

Guideline: To distinguish several declarations the special suffixes below **should be** used. The variables declared with the types are not concerned with these suffixes.

Type	**Letters**	**Suffix**	**Example**
typedef	upper/lowercase, "_"	_t	MyType_t, type_t
define	uppercase, "_"	_C	MY_DEFINE_C
enum members	Upper/lowercase, "_"	_E	MyEnumMember_E
Outdated form:			
enum	*lowercase, "_"*	*_e*	*my_enum_e*
struct	*lowercase, "_"*	*_s*	*my_struct_s*
union	*lowercase, "_"*	*_u*	*my_union_u*

There are some exceptions regarding the naming of defines: TRUE, FALSE, NULL and the comparison operators EQU, NEQ, OR, AND and NOT can be used without the suffix "_C".

It is recommended to write the name of the identifiers of the declarations like defined for each category in the table above. By mixing of upper/lowercases, the words should start with an uppercase followed by lowercase characters.

The forms of 'enum', 'struct' and 'union' without 'typedef' are outdated – they were 'typedefs' as well. Their naming convention is given only for informative reason and for compatibility with old software.

Reason: This increase readability and testability.

MISRA: n/a

Example: OK:

```
typedef (simple):
typedef unsigned long uint32_t;

struct:
typedef struct MyStruct_tag
{
    sint32_t lVal1;
    sint32_t lVal2;
} MyStruct_t;
:
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

```
MyStruct_t MyStructInstance;
```

union:

```
typedef union MyUnion_tag
{
    sint32_t lVal1;
    float32_t fVal1;
} MyUnion_t;
:
MyUnion_t MyUnionInstance;
```

enum:

```
typedef enum MyEnum_tag
{
    Value1_E = 0,
    Value2_E,
    Value3_E
} MyEnum_t;
:
MyEnum_t MyEnumInstance;
```

define:

```
#define MY_CONST_C (sint32_t)0xF7F7
```

UML-class:

In the C language, it can take the form of a struct, a union, an enum or a list of #defines. See examples above.

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

8.24 Type of a “#define” Value

ID: [LAT9-0201-00154]
[Type: Guideline]

Guideline: The type of each #define value **should be** given explicitly at its declaration via the cast form by including both the type and the value between braces.

Reason: A type-less value defined as alias may be interpreted differently depending on the environment (16 Bit, 32 Bit, etc.) and its condition of use. The precision of its type avoids any misunderstanding and unexpected behavior.

MISRA: n/a

Example: wrong:

```
#define NULL 0 /*Nb of Bits? Zero? Character \0? NULL Pointer?*/
#define MY_ALIAS_C 0x8000 /* 16Bits? 32Bits? 32768? -32768? */
```

OK:

```
#define NULL ((void*)0)
#define MY_ALIAS_C ((sint32)0x8000)
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

8.25 Active Code Hidden in a “#define”

ID: [LAT9-0201-00155]
[Type: Guideline]

Guideline: The expansion of simple #define-aliases (name without ‘( )’) **should not** produce directly executable code (active code), but only passive code like values, types, text-strings, etc.

For active code, uses either C regular-, C inline-functions or assembler functions instead #define. For use of C inline-functions, see restrictions in [LAT9-0201-00014] and [LAT9-0201-00074]. See also [LAT9-0201-00126] for assembler code.

The case of function-like C-Macro (#define with name including ‘( )’) is not covered by this guideline. See [LAT9-0201-00130] for this case.

Reason: Avoid the risk to have unwished and hard-recognizable side effects in C-code (e.g. access to registers which are also used by the compiler, dead-lock, etc.).

MISRA: n/a

Example: wrong:

```
#define  MY_MACRO \
{ \
   asm(" PUSH AL"); \
   asm(" MOV AL, #0xFFFF"); \
   asm(" RPT #3 || MOV *(0:0x0A98), AL"); \
   asm(" POP AL"); \
}

#define NOP asm("nop") /* Influence the Program Counter reg. */

#define WAITLOOP while(GlobVar_g >= 0)
```

OK:

```
#define NULL ((void*)0)
#define MY_ALIAS_C ((sint32)0x8000)
#define MY_ALIAS_STRING_C "Hello"
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

8.26 No Calculation Expression at Function Call

ID: [LAT9-0201-00113]
[Type: Guideline]

Guideline: Function parameters **should not** be calculated during the function call.

Reason: Testability and readability is improved. Comments can be better placed in the source code.

MISRA: n/a

Example: wrong:

```
Function1(nVar1 + TEN_C);
```

OK:

```
nVar2 = nVar1 + TEN_C;
Function1(nVar2);
```

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

9 Amendment Record

Issue	Date	Revision
0 - 10	1996 - 2018	See issue 10
11	11.06.2019	**Description of change:** 00-E00007052 SCS does not take into account applicable rules of D6-82071, D6-82952 Update of rules LAT9-0201-00006, LAT9-0201-00123 **Change category (minor/major):** Minor **Reason for the change:** - Process/change modification (including changes)

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

10 APPENDIX 1: Cross-Reference Matrix to MISRA-C

MISRA C Chapter	Rule-Number	Requirement	Level MISRA	Liebherr Coding Standard	Importancy Liebherr
1. Environment	1.1	All code shall conform to ISO 9899:1990 "Programming languages-C", amended and corrected by ISO/IEC 9899/COR1:1995, ISO/IEC9899/AMD1:1995, and ISO/IEC 9899/COR2:1996.	required	LAT9-0201-00014	Rule
	1.2	No reliance shall be placed on undefined or unspecified behavior.	required	LAT9-0201-00132 LAT9-0201-00051 (partly) LAT9-0201-00053 (partly) LAT9-0201-00073 (partly)	Rule Rule Rule Rule
	1.3	Multiple compilers and/or languages shall only be used if there is a common defined interface for object code to which the languages/compilers/assemblers conform	required	not covered (as C only HL language used)	Rule
	1.4	The compiler/linker shall be checked to ensure that 31 character significance and case sensitivity are supported for external identifiers.	required	not covered	
	1.5	Floating-point implementations should comply with a defined floating-point standard.	advisory	LAT9-0201-00153	Rule
2. Language Extension	2.1	Assembly language shall be encapsulated and isolated.	required	LAT9-0201-00126	Rule
	2.2	Source code shall only use /* ... */ style comments.	required	LAT9-0201-00006	Rule
	2.3	The character sequence /* shall not used within comment.	required	LAT9-0201-00124	Rule

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

MISRA C Chapter	Rule-Number	Requirement	Level MISRA	Liebherr Coding Standard	Importancy Liebherr
	2.4	Sections of code should not be "commented out".	advisory	LAT9-0201-00125	Guideline
3. Documentation	3.1	All usage of implementation-defined behavior shall be documented.	required	LAT9-0201-00152	Rule
	3.2	The character set and the corresponding encoding shall be documented.	required	LAT9-0201-00015	Rule
	3.3	The implementation of integer division in the chosen compiler should be determined, documented and taken into account.	advisory	not covered	
	3.4	All uses of the *#pragma* directive shall be documented and explained.	required	not covered	
	3.5	If it is being relied upon, the implementation-defined behavior and packing of bitfields shall be documented.	required	not covered	
	3.6	All libraries used in production code shall be written to comply with the provisions of this document, and shall have been subject to appropriate validation.	required	not covered	
4. Character Sets	4.1	Only those escape sequences that are defined in the ISO C standard shall be used.	required	LAT9-0201-00015	Rule
	4.2	Trigraphs shall not be used.	required	LAT9-0201-00020	Rule
5. Identifiers	5.1	Identifiers (internal and external) shall not rely on the significance of more than 31 characters.	required	not covered (compilers tested for compatibility)	Rule
	5.2	Identifiers in an inner scope shall not use the same name as an identifier in an outer scope, and therefore hide that identifier.	required	LAT9-0201-00093	Rule
	5.3	A *typedef* name shall be a unique identifier.	required	not covered	

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

MISRA C Chapter	Rule-Number	Requirement	Level MISRA	Liebherr Coding Standard	Importancy Liebherr
	5.4	A tag name shall be a unique identifier.	required	not covered	
	5.5	No object or function identifier with static storage duration should be reused.	advisory	not covered	
	5.6	No identifier in one name space should have the same spelling as an identifier in another name space, with the exception of structure and union member names.	advisory	not covered	
	5.7	No identifier name should be reused.	advisory	not covered	
6. Types	6.1	The plain *char* type shall be used only for the storage and use of character values.	required	LAT9-0201-00047 LAT9-0201-00150	Rule Guideline
	6.2	*signed* and *unsigned char* type shall be used only for the storage and use of numeric values.	required	LAT9-0201-00150	Guideline
	6.3	*typedefs* that indicate size and signedness should be used in place of the basic types.	advisory	LAT9-0201-00047	Rule
	6.4	Bit fields shall only be defined to be of type *unsigned int* or *signed int*.	required	LAT9-0201-00149 LAT9-0201-00150	Rule Guideline
	6.5	Bit fields of type *signed int* shall be at least 2 bits long.	required	LAT9-0201-00149	Rule
7. Constants	7.1	Octal constants (other than zero) and octal escape sequences shall not be used.	required	LAT9-0201-00146	Rule
8. Declarations and definitions	8.1	Functions shall have prototype declarations and the prototype shall be visible at both the function definition and call.	required	LAT9-0201-00016	Rule

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

MISRA C Chapter	Rule-Number	Requirement	Level MISRA	Liebherr Coding Standard	Importancy Liebherr
	8.2	Whenever an object or function is declared or defined, its type shall be explicitly stated.	required	LAT9-0201-00063	Rule
	8.3	For each function parameter the type given in declaration and definition shall be identical, and the return types shall also be identical.	required	LAT9-0201-00024	Rule
	8.4	If objects or functions are declared more than once their types shall be compatible.	required	not covered	
	8.5	There shall be no definitions of objects or functions in a header file.	required	LAT9-0201-00074	Rule
	8.6	Functions shall be declared at file scope.	required	LAT9-0201-00016	Rule
	8.7	Objects shall be defined at block scope if they are only accessed from within a single function.	required	not covered	
	8.8	An external object or function shall be declared in one and only one file.	required	LAT9-0201-00074	Rule
	8.9	An identifier within external linkage shall have exactly one external definition.	required	not covered	
	8.10	All declarations and definitions of objects or functions at file scope shall have internal linkage unless external linkage is required.	required	not covered	
	8.11	The *static* storage class specifier shall be used in definitions and declarations of objects and functions that have internal linkage.	required	not covered	
	8.12	When an array is declared with external linkage, its size shall be stated explicitly or defined implicitly by initialization.	required	LAT9-0201-00051	Rule
9. Initialization	9.1	All automatic variables shall have been assigned a value before being used.	required	LAT9-0201-00056	Rule
	9.2	Braces shall be used to indicate and match the structure in the non-zero initialization of arrays and structures.	required	LAT9-0201-00052	Rule

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

MISRA C Chapter	Rule-Number	Requirement	Level MISRA	Liebherr Coding Standard	Importancy Liebherr
	9.3	In an enumerator list, the "=" construct shall not be used to explicitly initialize members other than the first, unless all items are explicitly initialized.	required	LAT9-0201-00066	Rule
10. Arithmetic type conversion	10.1	The value of an expression of integer type shall not be implicitly converted to a different underlying type if: a) it is not a conversion to a wider integer type of the same signedness, or b) the expression is complex, or c) the expression is not constant and is a function argument, or d) the expression is not constant an is a return expression	required	LAT9-0201-00054	Rule
	10.2	The value of an expression of floating type shall not be implicitly converted to a different type if: a) it is not a conversion to a wider floating type, or b) the expression is complex, or c) the expression is a function argument, or d) the expression is a return expression.	required	LAT9-0201-00054	Rule
	10.3	The value of a complex expression of integer type may only be cast to a type that is narrower and of the same signedness as the underlying type of the expression.	required	LAT9-0201-00123	Rule
	10.4	The value of a complex expression of floating type may only be cast to a narrower floating type.	required	LAT9-0201-00123	Rule
	10.5	If the bitwise operators ~ and << are applied to an operand of underlying type *unsigned char* or *unsigned short*, the result shall be immediately cast to the underlying type of the operand.	required	LAT9-0201-00086	Rule
	10.6	A "U" suffix shall be applied to all constants of *unsigned* type.	required	LAT9-0201-00046	Rule
11. Pointer type conversion	11.1	Conversions shall not be performed between a pointer to a function and any type other than an integral type.	required	LAT9-0201-00055	Rule
	11.2	Conversions shall not be performed between a pointer to a object and any type other than an integral type, another pointer to object type or a pointer to *void.*	required	LAT9-0201-00055	Rule

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

MISRA C Chapter	Rule-Number	Requirement	Level MISRA	Liebherr Coding Standard	Importancy Liebherr
	11.3	A cast should not be performed between a pointer type and an integral type.	advisory	LAT9-0201-00055	Rule
	11.4	A cast should not be performed between a pointer to object type and a different pointer to object type.	advisory	LAT9-0201-00055	Rule
	11.5	A cast shall not be performed that removes any *const* or *volatile* qualification from the type addressed by a pointer.	required	LAT9-0201-00055	Rule
12. Expressions	12.1	Limited dependence should be placed on C's operator precedence rules in expressions.	advisory	LAT9-0201-00089	Rule
	12.2	The value of an expression shall be the same under any order of evaluation that the standard permits.	required	LAT9-0201-00042	Rule
	12.3	The *sizeof* operator shall not be used on expressions that contain side effects.	required	LAT9-0201-00085	Rule
	12.4	The right-hand operand of a logical && or || operator shall not contain side effects.	required	LAT9-0201-00137	Rule
	12.5	The operands of a logical && or || shall be *primary-expressions.*	required	LAT9-0201-00148	Rule
	12.6	The operands of logical operators (&&, || and !) should be effectively Boolean. Expressions that are effectively Boolean should not be used as operands to operators other than (&&, || and !).	advisory	not covered	
	12.7	Bitwise operators shall not be applied to operands whose underlying type is signed.	required	LAT9-0201-00151	Rule
	12.8	The right-hand operand of a shift operator shall lie between zero and one less than the width in bits of the underlying type of the left-hand operand.	required	LAT9-0201-00084	Rule
	12.9	The unary minus operator shall not be applied to an expression whose underlying type is unsigned.	required	LAT9-0201-00082	Rule
	12.10	The comma operator shall not be used.	required	LAT9-0201-00083	Rule

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

MISRA C Chapter	Rule-Number	Requirement	Level MISRA	Liebherr Coding Standard	Importancy Liebherr
	12.11	Evaluation of constant unsigned integer expressions should not lead to wrap-around.	advisory	LAT9-0201-00050	Guideline
	12.12	The underlying bit representations of floating-point values shall not be used.	required	LAT9-0201-00073	Rule
	12.13	The increment (++) and decrement (--) operators should not be mixed with other operators in an expression.	advisory	LAT9-0201-00042	Rule
13. Control statement expressions	13.1	Assignment operators shall not be used in expressions that yield a Boolean value.	required	LAT9-0201-00087	Rule
	13.2	Tests of a value against zero should be made explicit, unless the operand is effectively Boolean.	advisory	LAT9-0201-00037	Rule
	13.3	Floating-point expressions shall not be tested for equality or inequality.	required	LAT9-0201-00067	Rule
	13.4	The controlling expression of a *for* statement shall not contain any objects of floating type.	required	LAT9-0201-00122	Rule
	13.5	The three expressions of a *for* statement shall be concerned only with loop control.	required	LAT9-0201-00120	Rule
	13.6	Numeric variables being used within a *for* loop for iteration counting shall not be modified in the body of the loop.	required	LAT9-0201-00121	Rule
	13.7	Boolean operations whose results are invariant shall not be permitted.	required	LAT9-0201-00138	Rule
14. Control flow	14.1	There shall be no unreachable code.	required	LAT9-0201-00019	Rule
	14.2	All non-null statements shall either: a) have at least on side-effect however executed, or b) cause control flow to change.	required	LAT9-0201-00042	Rule
	14.3	Before preprocessing, a null statement shall only occur on a line by itself; it may be followed by a comment provided that the first character following the null statement is a white-space character.	required	LAT9-0201-00140	Rule

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

MISRA C Chapter	Rule-Number	Requirement	Level MISRA	Liebherr Coding Standard	Importancy Liebherr
	14.4	The *goto* statement shall not be used.	required	LAT9-0201-00032	Rule
	14.5	The *continue* statement shall not be used.	required	LAT9-0201-00041	Rule
	14.6	For any iteration statement there shall be at most one *break* statement used for loop termination.	required	LAT9-0201-00041	Rule
	14.7	A function shall have a single point of exit at the end of the function.	required	LAT9-0201-00133	Rule
	14.8	The statement forming the body of a *switch, while, do ... while* or *for* statement shall be a compound statement.	required	LAT9-0201-00011	Rule
	14.9	An *if* (expression) construct shall be followed by a compound statement. The *else* keyword shall be followed by either a compound statement, or another *if* statement.	required	LAT9-0201-00011	Rule
	14.10	All *if ... else* constructs shall be terminated with an *else* clause.	required	LAT9-0201-00039	Rule
15. Switch statements	15.1	A switch label shall only be used when the most closely-enclosing compound statement is the body of a *switch* statement.	required	LAT9-0201-00038	Rule
	15.2	An unconditional *break* statement shall terminate every non-empty switch clause.	required	LAT9-0201-00115	Rule
	15.3	The final clause of a *switch* statement shall be the *default* clause.	required	LAT9-0201-00114	Rule
	15.4	A *switch* expression shall not represent a value that is effectively Boolean.	required	LAT9-0201-00116	Rule
	15.5	Every *switch* statement shall have at least one *case* clause.	required	LAT9-0201-00114	Rule
16. Functions	16.1	Functions shall not be defined with a variable number of arguments.	required	not covered	

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

MISRA C Chapter	Rule-Number	Requirement	Level MISRA	Liebherr Coding Standard	Importancy Liebherr
	16.2	Functions shall not call themselves, either directly or indirectly.	required	not covered	
	16.3	Identifiers shall be given for all of the parameters in a function.	required	not covered	
	16.4	The identifiers used in the declaration and definition of a function shall be identical.	required	LAT9-0201-00024	Rule
	16.5	Functions with no parameters shall be declared with parameter type *void.*	required	LAT9-0201-00024	Rule
	16.6	The number of arguments passed to a function shall match the number of parameters.	required	LAT9-0201-00029	Rule
	16.7	A pointer parameter in a function prototype should be declared as pointer to *const* if the pointer is not used to modify the addressed object.	advisory	LAT9-0201-00102	Guideline
	16.8	All exit paths from a function with non-void return type shall have an explicit *return* statement with an expression.	required	LAT9-0201-00027	Rule
	16.9	A function identifier shall only be used with either a preceding &, or with a parenthesized parameter list, which may be empty.	required	LAT9-0201-00139	Rule
	16.10	If a function returns error information, then that error information shall be tested.	required	not covered	
17. Pointers and arrays	17.1	Pointer arithmetic shall only be applied to pointers that address an array or array element.	required	LAT9-0201-00059	Rule
	17.2	Pointer subtraction shall only be applied to pointers that address elements of the same array.	required	LAT9-0201-00059	Rule
	17.3	>,>=,<,<= shall not be applied to pointer types except where they point to the same array.	required	LAT9-0201-00060	Rule
	17.4	Array indexing shall be the only allowed form of pointer arithmetic.	required	LAT9-0201-00059	Rule

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

MISRA C Chapter	Rule-Number	Requirement	Level MISRA	Liebherr Coding Standard	Importancy Liebherr
	17.5	The declaration of objects should contain no more than 2 levels of pointer indirection.	advisory	LAT9-0201-00058	Rule
	17.6	The address of an object with automatic storage shall not be assigned to another object that may persist after the first object has ceased to exist.	required	not covered	
18. Structures and Unions	18.1	All structure an union types shall be complete at the end of a translation unit.	required	not covered	
	18.2	An object shall not be assigned to an overlapping object.	required	not covered	
	18.3	An area of memory shall not be reused for unrelated purposes.	required	not covered	
	18.4	Unions shall not be used.	required	not covered	
19. Preprocessor directives	19.1	*#include* statements in a file should only be preceded by other preprocessor directives or comments.	advisory	LAT9-0201-00044	Rule
	19.2	Non-standard characters should not occur in header file names in *#include* directives.	advisory	not covered	
	19.3	The *#include* directive shall be followed by either a <*filename*> or *"filename"* sequence.	required	LAT9-0201-00044	Rule
	19.4	C macros shall only expand to a braced initialiser, a constant, a parenthesized expression, a type qualifier, a storage class specifier, or a do-while-zero construct.	required	LAT9-0201-00021 (partly)	Rule
	19.5	Macros shall not be *#define*'d or *#undef*'d within a block.	required	LAT9-0201-00129	Rule
	19.6	*#undef* shall not be used.	required	LAT9-0201-00128	Rule
	19.7	A function should be used in preference to a function-like macro.	advisory	LAT9-0201-00130	Rule

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

MISRA C Chapter	Rule-Number	Requirement	Level MISRA	Liebherr Coding Standard	Importancy Liebherr
	19.8	A function-like macro shall not be invoked without all of its arguments.	required	LAT9-0201-00130 LAT9-0201-00157	Rule Rule
	19.9	Arguments to a function-like macro shall not contain tokens that look like preprocessing directives.	required	LAT9-0201-00130 LAT9-0201-00157	Rule Rule
	19.10	In the definition of a function-like macro each instance of a parameter shall be enclosed in parentheses unless it is used as the operand of # or ##.	required	LAT9-0201-00130 LAT9-0201-00157	Rule Rule
	19.11	All macro identifiers in preprocessor directives shall be defined before use, except in *#ifdef* and *#ifndef* preprocessor directives and the *defined()* operator.	required	LAT9-0201-00045	Rule
	19.12	There shall be at most one occurrence of the # or ## preprocessor operators in single macro definition.	required	LAT9-0201-00142	Rule
	19.13	The # and ## preprocessor operators should not be used.	advisory	LAT9-0201-00142	Rule
	19.14	The *defined* preprocessor operator shall only be used in one of the two standard forms.	required	LAT9-0201-00141	Rule
	19.15	Precautions shall be taken in order to prevent the contents of a header file being included twice.	required	LAT9-0201-00078	Rule
	19.16	Preprocessing directives shall be syntactically meaningful even when excluded by the preprocessor.	required	LAT9-0201-00143	Rule
	19.17	All *#else*, *#elif* and *#endif* preprocessor directives shall reside in the same file as the *#if* or *#ifdef* directive to which they are related.	required	LAT9-0201-00144	Rule

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

MISRA C Chapter	Rule-Number	Requirement	Level MISRA	Liebherr Coding Standard	Importancy Liebherr
20. Standard libraries	20.1	Reserved identifiers, macros and functions in the standard library, shall not be defined, redefined or undefined.	required	not covered	
	20.2	The names of standard library macros, objects and functions shall not be reused.	required	not covered	
	20.3	The validity of values passed to library functions shall be checked.	required	not covered	
	20.4	Dynamic heap memory allocation shall not be used.	required	not covered	
	20.5	The error indicator *errno* shall not be used.	required	not covered	
	20.6	The macro *offsetof*, in library *<STDDEF.H< i>, shall not be used.*	required	not covered	
	20.7	The *setjmp* macro and the *longjmp* function shall not be used.	required	not covered	
	20.8	The signal handling facilities of shall not be used.	required	not covered	
	20.9	The input/output library shall not be used in production code.	required	not covered	
	20.10	The library functions *atof, atoi* and *atol* from library shall not be used.	required	not covered	
	20.11	The library functions *abort, exit, getenv* and *system* from library shall not be used.	required	not covered	
	20.12	The time handling functions of library shall not be used.	required	not covered	
21. Run-time failures	21.1	Minimization of run-time failures shall be ensured by the use of at least one of: a) static analysis tools/techniques; b) dynamic analysis tools/techniques; c) explicit coding of checks to handle run-time faults.	required	not covered	

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

11 APPENDIX 2: Rules / Guidelines Summary

ID	Rule	Chapter	Type
LAT9-0201-00005	Layout of source file headers	8.1	Guideline
LAT9-0201-00006	Comments Style	7.1.1	Rule
LAT9-0201-00008	Indent every new block with 4 blanks (spaces)	8.2	Guideline
LAT9-0201-00010	Separate lines for the block borders "{" and "}"	8.21	Guideline
LAT9-0201-00011	"One-statement-block" block borders	7.1.3	Rule
LAT9-0201-00012	One statement per line	7.1.4	Rule
LAT9-0201-00013	Declaration of variables on separate lines	7.1.5	Rule
LAT9-0201-00014	Programming according to ANSI C definition	7.2.3	Rule
LAT9-0201-00015	Characters Sets	7.2.6	Rule
LAT9-0201-00016	Function prototyping	7.3.2.1	Rule
LAT9-0201-00019	Dead code	7.3.3.1	Rule
LAT9-0201-00020	Trigraphs	7.3.3.3	Rule
LAT9-0201-00021	Macro definition is restricted	7.3.2.2	Rule
LAT9-0201-00022	Maximum block depth limited to 6	8.13	Guideline
LAT9-0201-00024	Declaration of function parameters	7.3.4.1	Rule
LAT9-0201-00027	Needs of "return" statement in functions	7.3.4.2	Rule
LAT9-0201-00029	Function call parameters	7.3.4.5	Rule
LAT9-0201-00032	Use of "goto" and labels	7.3.3.4	Rule
LAT9-0201-00033	Expression bracket depth exceeding 7	8.7	Guideline
LAT9-0201-00034	Logical expressions enclosed in brackets	7.3.3.6	Rule
LAT9-0201-00036	Use restriction of assignment statement	7.3.3.7	Rule
LAT9-0201-00037	Test against zero	7.3.3.8	Rule
LAT9-0201-00038	"switch" statement scope for "case" and "default"	7.3.3.9	Rule

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

ID	Rule	Chapter	Type
LAT9-0201-00039	Usage and structure of "if-else if"-conditions	7.3.3.13	Rule
LAT9-0201-00040	Use of "for" loop counter	7.3.3.14	Rule
LAT9-0201-00041	Usage of break and continue statements in iterative structures	7.3.3.20	Rule
LAT9-0201-00042	Only one side effect or control flow change per non-null statement	7.3.3.21	Rule
LAT9-0201-00044	Preprocessor statement “#include”	7.3.3.29	Rule
LAT9-0201-00045	Preprocessor directive with identifiers	7.3.3.32	Rule
LAT9-0201-00046	Use of numeric constants	7.3.2.7	Rule
LAT9-0201-00047	Basic types	7.3.2.9	Rule
LAT9-0201-00048	Usage of static variables	7.3.2.10	Rule
LAT9-0201-00050	Overflow of unsigned integer with constant	8.16	Guideline
LAT9-0201-00051	Array declaration	7.3.2.11	Rule
LAT9-0201-00052	Array initialisation	7.3.2.12	Rule
LAT9-0201-00053	Array element control index	7.3.2.14	Rule
LAT9-0201-00054	Implicit type conversions	7.3.2.15	Rule
LAT9-0201-00055	Casting of pointers	7.3.2.17	Rule
LAT9-0201-00056	Variable and pointer initialization usage	7.3.2.18	Rule
LAT9-0201-00058	Complexity of pointer constructions limited to 2	7.3.2.19	Rule
LAT9-0201-00059	Pointer arithmetic	7.3.3.23	Rule
LAT9-0201-00060	Pointer comparison	7.3.3.24	Rule
LAT9-0201-00061	Function pointers	7.3.3.25	Rule
LAT9-0201-00063	Explicit type for declaration of Variables and Functions	7.3.2.20	Rule
LAT9-0201-00064	Structs and unions members	7.3.3.26	Rule
LAT9-0201-00066	Enum initialization	7.3.2.21	Rule
LAT9-0201-00067	Floating point comparisons	7.3.3.27	Rule
LAT9-0201-00068	Floating point integration	8.20	Guideline

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

ID	Rule	Chapter	Type
LAT9-0201-00073	Floating point representation	7.3.3.28	Rule
LAT9-0201-00074	Definition and declaration of functions or objects in include files	7.3.5.1	Rule
LAT9-0201-00075	‘Typedefs’ in include files	7.3.5.2	Rule
LAT9-0201-00078	Recursive/Repetitive inclusion of include files content	7.3.5.3	Rule
LAT9-0201-00080	Forbidden operators	7.3.6.1	Rule
LAT9-0201-00081	Use of ? : operator	7.3.6.2	Rule
LAT9-0201-00082	Use of the unary minus operator	7.3.6.3	Rule
LAT9-0201-00083	Use of the comma operator	7.3.6.4	Rule
LAT9-0201-00084	Use of the shift operator	7.3.6.5	Rule
LAT9-0201-00085	Use of the sizeof operator	7.3.6.6	Rule
LAT9-0201-00086	Bitwise operations must be cast	7.3.6.7	Rule
LAT9-0201-00087	Assignment operators	7.3.6.11	Rule
LAT9-0201-00089	Operator precedence	7.3.6.12	Rule
LAT9-0201-00091	Storage type "register"	8.12	Guideline
LAT9-0201-00093	Overlapping identifier name of outer scope	7.3.1.2	Rule
LAT9-0201-00094	Filenames	7.3.1.1	Rule
LAT9-0201-00095	Prefixes and postfixes of identifiers	8.8	Guideline
LAT9-0201-00097	Naming convention of identifiers	8.23	Guideline
LAT9-0201-00098	Significance of identifiers within the first 63 chars	7.3.1.3	Rule
LAT9-0201-00102	Pointer to Constant as function parameter	8.5	Guideline
LAT9-0201-00103	Indentation with tabulator	8.3	Guideline
LAT9-0201-00105	Horizontal Spacing	8.4	Guideline
LAT9-0201-00106	Location of function prototypes	8.9	Guideline
LAT9-0201-00107	Operation naming conventions for OOP	8.10	Guideline
LAT9-0201-00108	Maximum of 150 characters per line	8.11	Guideline
LAT9-0201-00109	Valid pointers	8.14	Guideline
LAT9-0201-00111	Array index type	7.3.2.13	Rule

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

ID	Rule	Chapter	Type
LAT9-0201-00113	No calculation expression at function call	8.27	Guideline
LAT9-0201-00114	Minimal "switch" statement	7.3.3.11	Rule
LAT9-0201-00115	Use of “break” in “switch / case” branches	7.3.3.10	Rule
LAT9-0201-00116	"switch" expression as Boolean value	7.3.3.12	Rule
LAT9-0201-00117	"for" loop must have an end condition	7.3.3.15	Rule
LAT9-0201-00118	Comparison operators for end condition in "for" loop	8.22	Guideline
LAT9-0201-00119	Variable used as end border in "for" loop	7.3.3.16	Rule
LAT9-0201-00120	Expressions used in "for" loop statements	7.3.3.17	Rule
LAT9-0201-00121	Change of loop counter of "for" loop	7.3.3.18	Rule
LAT9-0201-00122	No floating point type for counter of "for" loop	7.3.3.19	Rule
LAT9-0201-00123	Explicit type conversions with the "cast" operator	7.3.2.16	Rule
LAT9-0201-00124	No nested comments	7.1.2	Rule
LAT9-0201-00125	No source code “commented out”	8.15	Guideline
LAT9-0201-00126	Embedding assembler code into C code	7.3.3.5	Rule
LAT9-0201-00128	Undefine of macro is not allowed	7.3.2.3	Rule
LAT9-0201-00129	Scope of macro definition	7.3.2.4	Rule
LAT9-0201-00130	Function-like macros in handwritten code	7.3.2.5	Rule
LAT9-0201-00132	Reliance in undefined and unspecified behavior	7.2.4	Rule
LAT9-0201-00133	Number of "return" statements in a function	7.3.4.3	Rule
LAT9-0201-00134	Return value of “void” functions	7.3.4.4	Rule
LAT9-0201-00136	Complex type definition with tag	8.17	Guideline

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

ID	Rule	Chapter	Type
LAT9-0201-00137	Logical operators and side effects	7.3.6.9	Rule
LAT9-0201-00138	Conditions with invariant result	7.3.3.2	Rule
LAT9-0201-00139	Function call without defined parameters	7.3.4.6	Rule
LAT9-0201-00140	NULL statements	7.3.3.22	Rule
LAT9-0201-00141	Preprocessor statement ‘defined’	7.3.3.30	Rule
LAT9-0201-00142	Preprocessor operators # and ##	7.3.3.31	Rule
LAT9-0201-00143	Preprocessor directives always valid	7.3.3.33	Rule
LAT9-0201-00144	Preprocessor directives ‘#else’, ‘#elif’, ‘#endif’	7.3.3.34	Rule
LAT9-0201-00145	Preprocessor directive ‘#endif’ with comments	8.18	Guideline
LAT9-0201-00146	Octal constants	7.3.2.8	Rule
LAT9-0201-00148	Logical operators need primary expressions	7.3.6.10	Rule
LAT9-0201-00149	Bitfields types	7.3.2.22	Rule
LAT9-0201-00150	Definition of basic types	8.19	Guideline
LAT9-0201-00151	Bitwise operations only on unsigned integer	7.3.6.8	Rule
LAT9-0201-00152	Implementation-specific behaviours	7.2.5	Rule
LAT9-0201-00153	Floating-point implementations	7.2.2	Rule
LAT9-0201-00154	Type of a #define value	8.24	Guideline
LAT9-0201-00155	Active Code Hidden in a #define	8.25	Guideline
LAT9-0201-00157	Function-like macros in generated code	7.3.2.6	Rule
LAT9-0201-00158	Constant Pointer as function parameter	8.6	Guideline

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.

12 APPENDIX 3: DO-178B/DO-178C / ED-12B/ED-12C Objectives Coverage

This appendix summarizes with which rules or guidelines the objectives of the [DO-178B] / [DO-178C] are covered.

11.8a - Programming language(s) to be used and/or defined subset(s):

LAT9-0201-00014, LAT9-0201-00015, LAT9-0201-00132, LAT9-0201-00152, LAT9-0201-00126, LAT9-0201-00153, LAT9-0201-00020, LAT9-0201-00021, LAT9-0201-00128, LAT9-0201-00129, LAT9-0201-00130, LAT9-0201-00032, LAT9-0201-00036, LAT9-0201-00038, LAT9-0201-00115, LAT9-0201-00114, LAT9-0201-00116, LAT9-0201-00039, LAT9-0201-00040, LAT9-0201-00117, LAT9-0201-00119, LAT9-0201-00120, LAT9-0201-00121, LAT9-0201-00122, LAT9-0201-00041, LAT9-0201-00042, LAT9-0201-00140, LAT9-0201-00044, LAT9-0201-00141, LAT9-0201-00142, LAT9-0201-00045, LAT9-0201-00143, LAT9-0201-00047, LAT9-0201-00048, LAT9-0201-00051, LAT9-0201-00111, LAT9-0201-00061, LAT9-0201-00080, LAT9-0201-00081, LAT9-0201-00082, LAT9-0201-00083, LAT9-0201-00091, LAT9-0201-00068, LAT9-0201-00027, LAT9-0201-00133, LAT9-0201-00134, LAT9-0201-00139, LAT9-0201-00037, LAT9-0201-00046, LAT9-0201-00146, LAT9-0201-00053, LAT9-0201-00054, LAT9-0201-00123, LAT9-0201-00055, LAT9-0201-00059, LAT9-0201-00060, LAT9-0201-00063, LAT9-0201-00064, LAT9-0201-00066, LAT9-0201-00067, LAT9-0201-00073, LAT9-0201-00149, LAT9-0201-00084, LAT9-0201-00085, LAT9-0201-00086, LAT9-0201-00151, LAT9-0201-00137, LAT9-0201-00087, LAT9-0201-00050, LAT9-0201-00150, LAT9-0201-00118, LAT9-0201-00154, LAT9-0201-00113, LAT9-0201-00056, LAT9-0201-00102, LAT9-0201-00157, LAT9-0201-00158

11.8b - Source Code presentation standards:

LAT9-0201-00008, LAT9-0201-00012, LAT9-0201-00005, LAT9-0201-00006, LAT9-0201-00124, LAT9-0201-00011, LAT9-0201-00013, LAT9-0201-00103, LAT9-0201-00105, LAT9-0201-00108, LAT9-0201-00125, LAT9-0201-00145, LAT9-0201-00010, LAT9-0201-00034, LAT9-0201-00052, LAT9-0201-00148, LAT9-0201-00089

11.8c - Naming conventions for components, subprograms, variables, and constants:

LAT9-0201-00095, LAT9-0201-00097, LAT9-0201-00107, LAT9-0201-00093, LAT9-0201-00094, LAT9-0201-00098, LAT9-0201-00136

11.8d - Conditions and constraints imposed on permitted coding conventions:

LAT9-0201-00022, LAT9-0201-00033, LAT9-0201-00016, LAT9-0201-00019, LAT9-0201-00138, LAT9-0201-00024, LAT9-0201-00029, LAT9-0201-00058, LAT9-0201-00074, LAT9-0201-00075, LAT9-0201-00078, LAT9-0201-00106, LAT9-0201-00144, LAT9-0201-00109

11.8e - Constraints on the use of the coding tools:

All the rules are concerned due to their applicability for the different design methods SDDM1, SDDM2 and SDDM3 which are tool dependent.

CONFIDENTIAL

© Liebherr-Aerospace Lindenberg GmbH 2019. The reproduction, distribution and utilization of this document, as well as the communication of its contents to others without explicit authorization, is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design. The original released document is located in the PLM-System. Check the validity of the copy prior to use.